



**COORDENADORIA DE ENGENHARIA DA
COMPUTAÇÃO**

GUSTAVO AGUIAR BRANDÃO

LUCAS FOLTRAN

WILLIAN FERNANDES DE MORAES

**DESENVOLVIMENTO DE UMA APLICAÇÃO PARA GESTÃO E
VENDAS DE PRODUTOS SUSTENTÁVEIS**

SOROCABA, SP

2025



**GUSTAVO AGUIAR BRANDÃO
LUCAS FOLTRAN
WILLIAN FERNANDES DE MORAES**

**DESENVOLVIMENTO DE UMA APLICAÇÃO PARA GESTÃO E
VENDAS DE PRODUTOS SUSTENTÁVEIS**

Trabalho de conclusão de curso
apresentado ao Centro Universitário
Facens como exigência parcial para
obtenção do diploma de graduação em
Engenharia da Computação.

Orientador (a): Deivison Takatu.

SOROCABA, SP

2025

FICHA CATALOGRÁFICA ELABORADA PELA “BIBLIOTECA FACENS”

Bibliotecária responsável

Eliane da Rocha CRB 8062/8a

B817d

Brandão, Gustavo Aguiar.

Desenvolvimento de uma aplicação para gestão e vendas de produtos sustentáveis / por Gustavo Aguiar Brandão, Lucas Foltran Willian Fernandes de Moraes. - Sorocaba, SP: [s.n.], 2025.
73f.

Trabalho de Conclusão de Curso (Graduação) – Centro Universitário Facens – Coordenadoria de Engenharia da Computação - Curso de Engenharia da Computação, 2025.
Orientador: Prof. Me. Deivison Takatu.

1. Sustentabilidade. 2. CRM. 3. Consumo consciente. I. Foltran, Lucas. II. Moraes, Willian Fernandes de. III. Centro Universitário Facens. IV. Título.

CDD 621.39

**GUSTAVO AGUIAR BRANDÃO
LUCAS FOLTRAN
WILLIAN FERNANDES DE MORAES**

**DESENVOLVIMENTO DE UMA APLICAÇÃO PARA GESTÃO E
VENDAS DE PRODUTOS SUSTENTÁVEIS**

Trabalho de conclusão de curso apresentado
ao Centro Universitário Facens como
exigência parcial para obtenção do certificado
de conclusão do curso de Engenharia da
Computação.
Orientador (a): Deivison Takatu.

BANCA EXAMINADORA

Prof. Me Lucas Monteiro

Dedico este trabalho a todos que fizeram parte desta jornada de desenvolvimento profissional, e aos que contribuíram, direta ou indiretamente, para que meu objetivo fosse alcançado com sucesso, em meio a um processo permeado por alegrias.

AGRADECIMENTOS

Em primeiro lugar, expresso minha gratidão a Deus, fonte de sabedoria, força e perseverança, por ter iluminado meu caminho e me sustentado diante dos desafios desta trajetória acadêmica.

Aos meus familiares, registro meu reconhecimento pelo apoio incondicional, pelas palavras de incentivo e pela confiança depositada. O carinho e a presença de cada um foram fundamentais para que eu pudesse prosseguir com determinação e coragem.

Aos professores e orientador, agradeço pela dedicação, paciência e pelo compromisso em compartilhar conhecimentos que se tornaram pilares essenciais para o desenvolvimento deste trabalho. Suas orientações foram decisivas para o aprimoramento da pesquisa e para a construção do meu aprendizado.

À instituição de ensino, aos coordenadores, colegas de classe e demais colaboradores, manifesto minha sincera consideração pela contribuição direta e indireta no decorrer deste percurso, seja pelo convívio, pelo estímulo acadêmico ou pelo suporte prestado ao longo do curso.

A todos que, de forma direta ou indireta, contribuíram para a concretização deste projeto, deixo registrado meu profundo agradecimento.

RESUMO

O avanço da tecnologia tem transformado os processos comerciais e de relacionamento com clientes, demandando soluções cada vez mais eficientes para a gestão de vendas e fidelização. Nesse contexto, este trabalho propõe o desenvolvimento de uma aplicação baseada em *Customer Relationship Management* (CRM) Operacional integrada ao modelo *Software as a Service* (SaaS), voltada à gestão e comercialização de produtos sustentáveis. O estudo parte da necessidade de otimizar a organização interna de empresas do setor, melhorando o controle de clientes, pedidos e suporte, ao mesmo tempo em que contribui para a promoção do consumo consciente. Para tanto, utilizou-se como metodologia a revisão bibliográfica e o estudo de caso, aliados ao uso de tecnologias como *Angular*, *C#*, arquitetura MVC (*Model View Controller*), *Entity Framework* e banco de dados SQL, visando escalabilidade, segurança e eficiência. Os resultados esperados incluem maior eficiência na gestão de informações, melhoria na experiência do consumidor e incentivo à digitalização de empresas sustentáveis, fortalecendo a relação entre tecnologia e sustentabilidade.

Palavras-chave: Sustentabilidade. CRM Operacional. SaaS. Consumo Consciente.

ABSTRACT

The advancement of technology has transformed commercial processes and customer relationship management, creating a demand for increasingly efficient solutions for sales management and customer loyalty. In this context, this work proposes the development of an application based on Operational Customer Relationship Management (CRM) integrated with the Software as a Service (SaaS) model, focused on the management and commercialization of sustainable products. The study stems from the need to optimize the internal organization of companies in the sector, improving the control of clients, orders, and support, while also contributing to the promotion of conscious consumption. The methodology employed includes a literature review and a case study, combined with the use of technologies such as Angular, C#, MVC architecture, *Entity Framework*, and SQL database, aiming for scalability, security, and efficiency. The expected results include greater efficiency in information management, improvement in the consumer experience, and encouragement of digitalization in sustainable companies, strengthening the relationship between technology and sustainability.

Keywords: Sustainability. Operational CRM. SaaS. Conscious Consumption.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura MVC	25
Figura 2 – Representação gráfica SQL	27
Figura 3 – Versão Angular	30
Figura 4 – Arquitetura <i>back-end</i>	31
Figura 5 – Camada de Domínio	33
Figura 6 – Pacotes <i>NuGet</i>	34
Figura 7 – Criação do banco SQL	35
Figura 8 – Tabelas do sistema	36
Figura 9 – <i>String</i> de conexão	37
Figura 10 – Entidade Usuário	38
Figura 11 – Configuração Usuário	38
Figura 12 – Mapeamento	39
Figura 13 – Método de conexão	40
Figura 14 – Tabelas SQL	41
Figura 15 – HTML Angular	44
Figura 16 – Método de <i>Login</i>	45
Figura 17 – Serviço de <i>login</i>	46
Figura 18 – Lista de produtos HTML	47
Figura 19 – Carrinho de compras	48
Figura 20 – Envio do pedido HTML	49
Figura 21 – Montagem do pedido	50
Figura 22 – Envio do pedido	51
Figura 23 – Salvar pedido	52
Figura 24 – Interface de pagamento	53
Figura 25 – Início do pagamento	54
Figura 26 – Atualização do lead	55
Figura 27 – Atualização do pedido	56
Figura 28 – Fluxograma de navegação	58
Figura 29 – Cadastro	59
Figura 30 – E-mail de boas vindas	60
Figura 31 – Produtos	61

Figura 32 – Pagamento	61
Figura 33 – Compra Finalizada	62
Figura 34 – Acesso <i>Admin</i>	63
Figura 35 – E-mail novo cliente	64
Figura 36 – E-mail Pagamento	65

LISTA DE SIGLAS

API	Application Programming Interface (Interface de Programação de Aplicações)
ASP.NET	Active Server Pages .NET (Framework da Microsoft para desenvolvimento web)
B2B	Business-to-Business (Negócios entre Empresas)
C#	C Sharp (Linguagem de Programação Orientada a Objetos da Microsoft)
CLV	Customer Lifetime Value (Valor do Tempo de Vida do Cliente)
CRM	Customer Relationship Management (Gerenciamento de Relacionamento com o Cliente)
DCL	Data Control Language (Linguagem de Controle de Dados)
DDL	Data Definition Language (Linguagem de Definição de Dados)
DML	Data Manipulation Language (Linguagem de Manipulação de Dados)
DQL	Data Query Language (Linguagem de Consulta de Dados)
DTL	Data Transaction Language (Linguagem de Transação de Dados)
ERP	Enterprise Resource Planning (Planejamento de Recursos Empresariais)
HTML	HyperText Markup Language (Linguagem de Marcação de Hipertexto)
IDE	Integrated Development Environment (Ambiente de Desenvolvimento Integrado)
JSON	JavaScript Object Notation (Notação de Objeto JavaScript)
JWT	JSON Web Token (Token Web em Formato JSON)
LINQ	Language Integrated Query (Consulta Integrada à Linguagem)
MVC	Model View Controller (Padrão de Arquitetura que Separa Dados, Interface e Controle)
ORM	Object-Relational Mapping (Mapeamento Objeto-Relacional)
SaaS	Software as a Service (Software como Serviço)
SPA	Single Page Application (Aplicação de Página Única)
SQL	Structured Query Language (Linguagem de Consulta Estruturada)
SSMS	SQL Server Management Studio (Ambiente de Gerenciamento do SQL Server)
URL	Uniform Resource Locator (Localizador Padrão de Recursos)

SUMÁRIO

1 INTRODUÇÃO.....	13
2 O PAPEL DA TECNOLOGIA NA GESTÃO COMERCIAL.....	16
2.1 Customer Relationship Management	17
2.1.1 Tipos de CRM	19
2.1.2 Benefícios do CRM.....	21
3 DESENVOLVIMENTO.....	24
3.1 Tecnologias e Arquiteturas Utilizadas	24
3.1.1 Arquitetura MVC	24
3.1.2 Banco de Dados SQL.....	26
3.1.3 <i>Entity Framework</i>	28
3.1.4 <i>Angular</i>	29
3.2 Processo de desenvolvimento.....	30
3.2.1 Criação e configuração do projeto.....	31
3.2.2 Criação do banco e tabelas SQL.....	35
3.2.3 Criação do projeto <i>Angular</i>	42
3.2.3.1 Criação do componente de <i>login</i>	43
3.2.3.2 Montagem do pedido.....	47
3.2.3.3 Pagamento	52
4 RESULTADOS OBTIDOS.....	58
4.1 Fluxo Cliente	58
4.2 Fluxo Administrativo	62
5 CONSIDERAÇÕES FINAIS.....	66
5.1 Avaliação dos Resultados e Contribuições	66
5.2 Lições Aprendidas e Desenvolvimento de Competências.....	67
5.3 Recomendações para Trabalhos Futuros	67
REFERÊNCIAS.....	69
APÊNDICE A – REPOSITÓRIO DO PROJETO.....	72

1 INTRODUÇÃO

O avanço da tecnologia tem impactado significativamente diversos setores da sociedade, incluindo o comércio e a gestão de negócios. Com a crescente necessidade de empresas aprimorarem seus processos de vendas e relacionamento com clientes, surgem soluções tecnológicas voltadas para a organização e gerenciamento dessas interações. Entre essas soluções, destacam-se os sistemas de *Customer Relationship Management* (CRM), que auxiliam no controle e na análise de dados para otimizar o atendimento ao cliente.

No contexto da sustentabilidade, observa-se uma demanda crescente por produtos ecologicamente responsáveis, impulsionada pela conscientização ambiental e pela busca por práticas de consumo mais equilibradas. Esse cenário exige que empresas do setor sustentável adotem ferramentas capazes de organizar seus processos internos e, ao mesmo tempo, potencializar o impacto positivo que geram na sociedade e no meio ambiente. Dessa forma, sistemas que integram CRM e tecnologias web tornam-se essenciais não apenas para aprimorar a gestão comercial, mas também para fortalecer cadeias produtivas sustentáveis, ampliar o alcance de produtos de baixo impacto ambiental e incentivar hábitos de consumo conscientes. Este trabalho desenvolve uma aplicação para a gestão e vendas de produtos sustentáveis, integrando conceitos de CRM Operacional a um *Software as a Service* (SaaS) para facilitar o gerenciamento de clientes, pedidos e suporte. Além disso, a aplicação contribui para práticas sustentáveis ao reduzir retrabalhos, minimizar desperdícios operacionais, otimizar o uso de recursos e ampliar o acesso dos consumidores a produtos de menor impacto ambiental.

Apesar da crescente digitalização do setor comercial, muitas empresas que comercializam produtos sustentáveis ainda enfrentam dificuldades na organização de seus processos internos. A falta de um sistema eficiente pode resultar em falhas no atendimento, dificuldades na fidelização de clientes e baixa otimização dos dados coletados. Dessa forma, o desenvolvimento de um sistema que integre funcionalidades de um CRM Operacional contribui para a melhoria da relação entre empresas e consumidores.

O problema central deste estudo está na necessidade de otimizar a gestão de micro e pequenas empresas que atuam no mercado de produtos sustentáveis, muitas

vezes com recursos tecnológicos limitados e processos administrativos manuais. Essas organizações enfrentam desafios na organização de cadastros, controle de pedidos e fidelização de clientes, o que compromete a eficiência operacional e a qualidade do atendimento. Assim, este trabalho propõe o desenvolvimento de uma aplicação que integre funcionalidades de um CRM Operacional voltado a esse contexto, oferecendo um sistema acessível, escalável e de fácil utilização. A questão que norteia a pesquisa é: como um sistema de CRM Operacional, desenvolvido com tecnologias web, pode contribuir para a melhoria do gerenciamento e da experiência do consumidor em empresas que comercializam produtos sustentáveis?

O principal objetivo deste trabalho é desenvolver e avaliar uma aplicação web baseada em CRM Operacional voltada à gestão de vendas de produtos sustentáveis, buscando aprimorar os processos de relacionamento com o cliente, controle de pedidos e organização interna das empresas. O sistema proposto visa oferecer uma solução tecnológica acessível e escalável, que possibilite às micro e pequenas empresas do setor sustentável gerenciar de forma mais eficiente suas operações comerciais e, simultaneamente, maximizar o impacto positivo associado à comercialização de produtos sustentáveis, fortalecendo o consumo consciente.

Do ponto de vista metodológico, esta pesquisa caracteriza-se como qualitativa e aplicada, uma vez que busca compreender e propor soluções práticas para a gestão comercial de empresas que atuam com produtos sustentáveis. O método utilizado é o estudo de caso, permitindo a análise detalhada do desenvolvimento e da aplicação do sistema proposto em um contexto real. Como procedimentos técnicos, foram adotadas a revisão bibliográfica, para embasar teoricamente o estudo, a prototipagem, voltada ao desenvolvimento da aplicação, e a validação funcional, realizada por meio de testes práticos.

A aplicação foi desenvolvida utilizando *Angular* para o *front-end*, *C#* com arquitetura MVC na construção da API (*Application Programming Interface*) e *Entity Framework* para a comunicação com o banco de dados SQL. Foram utilizadas referências acadêmicas e materiais técnicos sobre CRM, SaaS e desenvolvimento de *software*, com o objetivo de fundamentar o processo de construção e validação do sistema proposto.

Os principais autores que embasam este estudo incluem as pesquisas de Simões (2021), Becker (2024) e Borsatto e Machado (2024), que oferecem uma base sólida sobre os conceitos e aplicações do CRM, abordando desde a sua implementação

até seus impactos na gestão empresarial. Além disso, foram consultadas as obras de Matiello (2022) e Zanatta (2024), que apresentam estudos de caso e análises sobre a utilização de sistemas CRM em diferentes contextos, contribuindo para a fundamentação teórica e prática deste trabalho.

A pesquisa foi estruturada com base em um levantamento bibliográfico detalhado, análise de sistemas existentes no mercado e desenvolvimento de um protótipo funcional. A estratégia metodológica incluiu a comparação de abordagens utilizadas por empresas do setor e a aplicação de conceitos estudados ao longo do curso.

Este trabalho está estruturado em cinco capítulos. No primeiro, apresenta-se a introdução ao tema, destacando a relevância da pesquisa e seus objetivos. O segundo capítulo discute o referencial teórico, reunindo os principais conceitos sobre CRM, SaaS e desenvolvimento web. O terceiro descreve a metodologia aplicada, com ênfase nas tecnologias e etapas do desenvolvimento. O quarto capítulo apresenta os resultados alcançados e, por fim, o quinto reúne as considerações finais, consolidando as conclusões do estudo.

Do ponto de vista científico, o trabalho se justifica pela crescente importância da transformação digital e da gestão de relacionamento com clientes (CRM) como instrumentos de modernização empresarial. Segundo Becker (2024), o uso de tecnologias de CRM impulsiona o sucesso comercial e fortalece o relacionamento entre empresas e consumidores, evidenciando seu papel estratégico na eficiência organizacional. Já Zanatta (2024) destaca os desafios enfrentados pelas empresas na implementação de sistemas de CRM e na integração dessas ferramentas à gestão de vendas, o que demonstra a necessidade de novas abordagens e soluções práticas.

Dessa forma, este estudo torna-se relevante ao propor o desenvolvimento de uma aplicação que une CRM Operacional e gestão sustentável de produtos, contribuindo para a modernização, eficiência operacional e promoção de práticas de consumo conscientes, reforçando o papel da tecnologia como aliada da sustentabilidade empresarial.

2 O PAPEL DA TECNOLOGIA NA GESTÃO COMERCIAL

Antes mesmo da adoção de ferramentas tecnológicas avançadas, as empresas que buscavam excelência na gestão comercial precisavam considerar aspectos estratégicos que vão além do lucro imediato, incluindo sustentabilidade e responsabilidade socioambiental. Segundo Renata (2021), práticas que equilibram desempenho econômico, responsabilidade social e preservação ambiental permitem atender às necessidades do presente sem comprometer as gerações futuras.

Para Das e Hassan (2021), a integração de estratégias de sustentabilidade corporativa contribui para maior eficiência operacional, redução de custos e fortalecimento da reputação organizacional. Lessa (2020) complementa que a implementação de práticas socioambientais, como reciclagem, economia de energia e ecoeficiência, promove a conscientização da população e reforça a competitividade das empresas, criando valor tanto para a sociedade quanto para o negócio.

A tecnologia aplicada à gestão comercial, especialmente por meio de sistemas como o CRM (*Customer Relationship Management*), desempenha papel essencial nesse contexto, ao organizar informações e apoiar decisões estratégicas que permitem alinhar objetivos econômicos com práticas social e ambientalmente responsáveis. Conforme destaca Renata (2021), o uso inteligente da tecnologia possibilita decisões mais conscientes, integrando rentabilidade e competitividade com responsabilidade socioambiental.

De acordo com Das e Hassan (2021), a sustentabilidade, quando combinada com sistemas tecnológicos de gestão, constitui um diferencial estratégico capaz de fortalecer a fidelização de clientes, aumentar a eficiência operacional e gerar vantagem competitiva sustentável. Para Lessa (2020), práticas de ecoeficiência e coleta seletiva, aliadas ao desenvolvimento tecnológico, contribuem para a redução do consumo de recursos não renováveis e para a criação de uma consciência ecológica mais ampla na sociedade.

Dessa forma, o uso da tecnologia na gestão comercial não se limita à automação de processos, mas se configura como um elemento integrador entre eficiência empresarial e responsabilidade socioambiental, criando valor compartilhado para empresas, clientes e comunidade.

A tecnologia se consolidou como um dos pilares essenciais para o sucesso comercial das empresas. Seu papel vai além da automação de tarefas operacionais,

ela fornece inteligência estratégica por meio de dados, integra setores e permite que a tomada de decisão seja embasada em informações precisas e em tempo real.

Nesse contexto, ferramentas como o CRM ganharam protagonismo. Através de sistemas integrados de gestão do relacionamento com o cliente, as empresas conseguem organizar, centralizar e analisar dados que antes estavam dispersos ou indisponíveis. Isso proporciona agilidade, personalização no atendimento e maior assertividade nas estratégias de venda.

A partir da análise de interações e preferências dos clientes, é possível compreender com mais profundidade seu comportamento, prever necessidades e criar ofertas sob medida. Tudo isso fortalece o relacionamento entre empresa e consumidor e aumenta consideravelmente a taxa de conversão de *leads* em clientes ativos. Além disso, os sistemas CRM se mostram como uma ponte entre o setor comercial e os demais setores da empresa, promovendo a integração de informações e esforços.

2.1 Customer Relationship Management

O Gerenciamento de Relacionamento com o Cliente, é mais do que uma solução tecnológica, trata-se de uma abordagem estratégica focada em criar, manter e aprimorar as relações com os clientes. Em um mercado altamente competitivo e digitalizado, o CRM se tornou um recurso fundamental para garantir a sustentabilidade e impulsionar o crescimento das empresas.

Ele permite uma visão completa do histórico de interações com cada cliente, reunindo dados que servem de base para decisões mais inteligentes, ações de marketing mais eficazes e um atendimento mais próximo e relevante. Como resultado, as empresas conseguem não só fidelizar seus clientes, mas também fortalecer sua posição no mercado por meio de uma gestão mais centrada nas pessoas.

Segundo Becker (2024), a tecnologia tem impulsionado significativamente o sucesso comercial das organizações, sendo o CRM uma das soluções mais eficazes nesse processo. O sistema de CRM armazena dados detalhados sobre os clientes, como nome, endereço, telefone, e também registra interações como chamadas, e-mails, visitas a websites, preferências e histórico de compras. Essas informações são fundamentais para que a empresa desenvolva um atendimento personalizado, proativo e baseado em dados reais. Ainda conforme Becker (2024, p. 135), o CRM vai muito além de uma simples lista de contatos: ele é uma plataforma integradora que reúne

dados valiosos, capacitando as equipes de vendas, marketing e suporte a oferecerem experiências mais coerentes e personalizadas ao consumidor. Dessa forma, o CRM colabora diretamente para o aumento da satisfação e fidelização do cliente, impactando positivamente nos resultados comerciais da empresa. Além disso, destaca que o CRM tem duas origens principais: uma tecnológica, oriunda da evolução das ferramentas de marketing, e outra gerencial, relacionada à crescente demanda por sistemas de gestão empresarial. A união desses dois elementos permite que o CRM não seja apenas uma ferramenta operacional, mas uma estratégia de relacionamento contínuo e inteligente.

O CRM moderno atua como um verdadeiro hub de relacionamento, centralizando dados e facilitando a comunicação entre diferentes departamentos da empresa. Isso proporciona uma visão unificada do cliente, permitindo que a organização ofereça experiências mais consistentes e alinhadas às expectativas do consumidor. A personalização dos atendimentos, baseada em informações históricas e comportamentais, transforma cada interação em uma oportunidade de fortalecer o vínculo com o cliente.

Além disso, o uso estratégico do CRM possibilita a identificação de padrões de comportamento, antecipação de necessidades e segmentação de públicos, o que aumenta significativamente a eficácia das campanhas de marketing e das abordagens de vendas. A automação de tarefas operacionais, como o envio de e-mails, notificações e atualizações de status, também contribui para a produtividade das equipes e para a redução de erros operacionais.

Outro ponto importante é a integração entre o CRM e outras tecnologias de gestão empresarial, como sistemas ERP, plataformas de e-commerce, redes sociais e ferramentas de análise de dados. Essa integração amplia a capacidade de análise e monitoramento do desempenho das ações comerciais, fornecendo indicadores essenciais para a tomada de decisões mais precisas e assertivas.

Por fim, vale destacar que o sucesso da implementação de um sistema CRM depende não apenas da tecnologia adotada, mas também do alinhamento cultural da empresa com uma mentalidade centrada no cliente. Isso envolve treinamentos, mudanças nos processos internos e o comprometimento das lideranças em adotar práticas orientadas à construção de relacionamentos duradouros.

2.1.1 Tipos de CRM

A Gestão de Relacionamento com o Cliente é uma das estratégias mais relevantes no ambiente empresarial contemporâneo. Frente à crescente complexidade das interações entre empresa e consumidor, e à enorme quantidade de dados gerados a cada contato, a organização que deseja se manter competitiva precisa investir em métodos estruturados de coleta, análise e utilização dessas informações. Como destaca Simões (2021), o CRM representa não apenas uma ferramenta tecnológica, mas uma filosofia empresarial focada no cliente, exigindo integração entre setores, inteligência de dados e ações personalizadas.

No entendimento de Silveira (2022), o CRM é essencial para transformar dados em conhecimento e ações eficazes, promovendo a fidelização do consumidor e o aumento da lucratividade da empresa. Em sua concepção, o CRM está apoiado em três pilares principais: CRM Operacional, CRM Analítico e CRM Colaborativo. Além desses, a crescente relevância das mídias digitais trouxe à tona uma quarta categoria: o CRM Social.

O CRM Operacional constitui a base funcional das atividades de relacionamento com o cliente. Ele atua diretamente nas interfaces de contato, como atendimento ao cliente, vendas, marketing e pós-venda, permitindo registrar, gerenciar e automatizar interações. Para Simões (2021), trata-se da camada responsável por garantir que cada ponto de contato com o consumidor seja documentado e acessível, criando uma linha do tempo do relacionamento.

Silveira (2022) salienta que, por meio do CRM Operacional, empresas conseguem direcionar chamados aos setores corretos, acelerar processos de suporte e armazenar o histórico de interações, construindo uma visão unificada do cliente. Por exemplo, uma empresa que utiliza um sistema CRM pode registrar cada atendimento feito por telefone, e quando o mesmo cliente entra em contato por e-mail, o atendente já possui todos os dados do último atendimento, facilitando uma resposta mais rápida e personalizada.

Essa abordagem promove eficiência operacional, reduz o retrabalho e fortalece a imagem da empresa perante o consumidor, já que ele percebe consistência e prontidão nos canais de atendimento. Além disso, possibilita a integração com outras plataformas, como ERPs e sistemas de logística, potencializando a automação de processos internos.

O CRM Analítico tem como principal objetivo transformar dados brutos em informações estratégicas. Ele analisa os registros de interações e comportamentos dos clientes para segmentá-los, prever tendências e recomendar ações de marketing, vendas ou fidelização. Simões (2021) aponta que esse tipo de CRM é especialmente útil para o desenvolvimento de estratégias orientadas por dados, pois possibilita a compreensão do comportamento dos clientes e o direcionamento de ações customizadas.

Silveira (2022), afirma que o CRM Analítico é capaz de identificar diferentes perfis de clientes e entender suas necessidades específicas. Dessa forma, uma empresa que sabe que um grupo de clientes compra frequentemente produtos sustentáveis pode direcionar promoções específicas para este público, com linguagem e ofertas personalizadas.

A aplicação de tecnologias como *Big Data*, *Business Intelligence* e *Machine Learning* tem tornado o CRM Analítico ainda mais poderoso. Com essas ferramentas, as empresas conseguem prever a rotatividade de clientes, estimar o valor de tempo de vida de um consumidor (CLV – *Customer Lifetime Value*) e até mesmo antecipar o melhor momento para lançar um novo produto ou serviço.

Além de contribuir para a personalização de campanhas, o CRM Analítico possibilita a avaliação da eficácia das estratégias utilizadas, permitindo ajustes mais rápidos e decisões baseadas em evidências concretas, em vez de intuições ou achismos.

O CRM Colaborativo é o elo que conecta as informações extraídas do CRM Operacional e Analítico com as diferentes áreas e stakeholders da organização. Para Simões (2021), ele garante que todos os departamentos envolvidos no relacionamento com o cliente compartilhem dados e trabalhem com base em um objetivo comum: oferecer uma experiência coesa, ágil e de alto valor percebido.

Silveira (2022), define o CRM Colaborativo como a integração digital de todas as áreas da empresa voltadas ao relacionamento com o cliente, inclusive parceiros externos. Isso significa que marketing, comercial, financeiro e atendimento, por exemplo, podem acessar os mesmos dados, evitar ruídos de comunicação e trabalhar em convergência estratégica.

A colaboração também pode ocorrer fora da organização, com fornecedores, distribuidores e representantes comerciais, criando um ecossistema interligado e orientado ao cliente. Isso é especialmente relevante em mercados B2B (*business-to-*

business), onde o relacionamento é complexo, de longo prazo e envolve múltiplos tomadores de decisão.

O advento da internet e das redes sociais transformou radicalmente a forma como empresas e clientes se relacionam. Nesse cenário, surge o CRM Social, uma evolução do modelo tradicional, que considera os dados e interações provenientes de plataformas como Facebook, Instagram, Twitter e WhatsApp.

Para Silveira (2022), o CRM Social permite que a empresa acompanhe conversas, comentários e menções em tempo real, interaja diretamente com o cliente e capte insights sobre hábitos, preferências e insatisfações de forma espontânea. Conforme Greenberg (2001), citado pelo autor, o CRM Social cria um ambiente colaborativo e transparente, em que o cliente se torna parte ativa da construção da marca.

Essa modalidade de CRM também funciona como ferramenta de gestão de reputação, monitorando crises, respondendo rapidamente a críticas e reforçando os pontos fortes da empresa. Além disso, permite o enriquecimento de perfis com informações públicas disponíveis nas redes sociais, tornando as campanhas de marketing ainda mais direcionadas.

Organizações que adotam estratégias de CRM Social tendem a ter maior engajamento, fidelização e recomendação espontânea, pois criam laços baseados em confiança, empatia e presença ativa nas plataformas digitais que os consumidores mais utilizam.

2.1.2 Benefícios do CRM

A adoção de um sistema CRM oferece uma série de benefícios estratégicos para as empresas, especialmente no contexto de um mercado cada vez mais competitivo e orientado por dados. Entre os principais benefícios do CRM, podemos destacar:

Melhora no Atendimento ao Cliente: O CRM permite que as empresas ofereçam um atendimento mais personalizado e eficiente. Com acesso rápido ao histórico de interações do cliente, os atendentes podem oferecer soluções mais rápidas e eficazes, o que resulta em maior satisfação e fidelização.

Aumento da Eficiência Operacional: A automação de tarefas repetitivas e processos de negócios, como o envio de e-mails, atualizações de status e gerenciamento de contatos, pode reduzir significativamente a carga de trabalho

manual. Isso permite que as equipes se concentrem em atividades mais estratégicas, aumentando a produtividade.

Melhor Gestão de Relacionamentos: O CRM centraliza as informações dos clientes, permitindo uma visão única e detalhada de cada relacionamento. Isso facilita a segmentação de clientes e o desenvolvimento de ofertas personalizadas, aumentando a probabilidade de conversão e retenção de clientes.

Aprimoramento das Estratégias de Vendas: Os sistemas de CRM ajudam as equipes de vendas a acompanhar o progresso dos *leads*, identificar oportunidades de *upsell* ou *cross-sell*, e prever quais clientes têm maior potencial de compra. Com isso, as abordagens de vendas tornam-se mais assertivas, aumentando as taxas de fechamento.

Acesso a Relatórios e Análises em Tempo Real: O CRM permite que as empresas obtenham dados em tempo real sobre o comportamento dos clientes, desempenho de vendas e eficiência de campanhas. Com esses dados, é possível tomar decisões mais informadas e ajustar estratégias com rapidez, com base em dados concretos.

Fidelização de Clientes: Ao promover um atendimento de alta qualidade e personalização, o CRM contribui para a construção de um relacionamento duradouro com os clientes. Clientes satisfeitos são mais propensos a se tornar leais à marca e a recomendar a empresa para outros, gerando um ciclo de crescimento orgânico.

Integração entre Departamentos: O CRM integra os diferentes setores da empresa (vendas, marketing, atendimento ao cliente), permitindo um fluxo de informações contínuo e colaborativo. Isso melhora a comunicação interna, evita falhas de comunicação e garante que todos os departamentos trabalhem em direção aos mesmos objetivos estratégicos.

Redução de Custos: A automação e a centralização de dados podem reduzir os custos operacionais relacionados a processos manuais e à perda de informações. Além disso, a análise preditiva do CRM pode ajudar a evitar investimentos em clientes ou campanhas que provavelmente não trarão retorno, otimizando os recursos da empresa.

Aumento da Personalização nas Interações: Ao integrar dados históricos e comportamentais dos clientes, o CRM permite a criação de campanhas de marketing e atendimento totalmente personalizadas. Isso não só aumenta a satisfação do cliente, mas também melhora as taxas de conversão e a efetividade das campanhas.

Facilidade na Identificação de Oportunidades de Negócio: Com o CRM, é

possível analisar padrões e tendências no comportamento dos consumidores, identificando oportunidades de negócio que podem ser exploradas. Isso permite que a empresa reaja rapidamente a novas demandas do mercado e se posicione estrategicamente.

3 DESENVOLVIMENTO

O desenvolvimento da aplicação teve como objetivo criar uma solução prática, eficiente e funcional para a gestão e comercialização de produtos sustentáveis, integrando conceitos de CRM Operacional. Nesta fase, foram adotadas tecnologias modernas que garantissem segurança, boa performance e uma experiência de usuário intuitiva, atendendo às necessidades operacionais e estratégicas das empresas.

3.1 Tecnologias e Arquiteturas Utilizadas

Para viabilizar essas características, foram escolhidas tecnologias e arquiteturas que facilitassem a manutenção e a integração entre *front-end* e *back-end*. Entre os principais elementos estão a arquitetura MVC (*Model-View-Controller*), o banco de dados SQL, o *Entity Framework* para mapeamento objeto-relacional e o *Angular* framework para a criação da interface do usuário, permitindo um sistema modular e adaptável às demandas do mercado de produtos sustentáveis.

3.1.1 Arquitetura MVC

O padrão *Model-View-Controller* (MVC) é uma arquitetura de *software* que organiza a aplicação em três componentes distintos: Modelo (*Model*), Visão (*View*) e Controlador (*Controller*). Segundo Luciano e Alves (2011), o MVC surgiu com o objetivo de separar a camada de apresentação da lógica de negócio, promovendo modularidade, facilidade de manutenção e reutilização de código em diferentes contextos de aplicação.

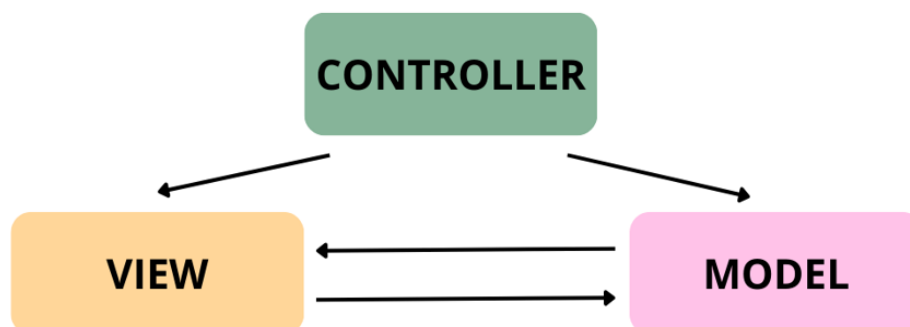
No padrão MVC, o Modelo (*Model*) é responsável por armazenar e gerenciar o estado da aplicação, incluindo dados e regras de negócio, além de interagir com o banco de dados sempre que necessário. A Visão (*View*) tem a função de apresentar essas informações ao usuário de forma clara e dinâmica, proporcionando uma interface interativa que reflete as alterações no modelo em tempo real. Por sua vez, o Controlador (*Controller*) atua como intermediário, interpretando as ações do usuário e coordenando a execução das operações no modelo, garantindo que o fluxo de informações entre a camada de apresentação e a lógica de negócio seja organizado, eficiente e consistente. O MVC é especialmente útil em aplicações web, permitindo que diferentes interfaces

sejam desenvolvidas sem alterar a lógica de negócio. Historicamente, antes do MVC, as aplicações eram monolíticas ou em duas camadas, com acoplamento elevado entre apresentação, regras de negócio e acesso a dados. A adoção do MVC proporciona baixo acoplamento e alta coesão, aumentando a flexibilidade para manutenção e evolução do sistema (Luciano & Alves, 2011).

No contexto deste trabalho, a aplicação da arquitetura MVC no *ASP.NET Core* possibilitou estruturar o *back-end* de forma modular, integrando de maneira eficiente o *Entity Framework* e garantindo que a aplicação pudesse ser facilmente conectada ao *front-end* em *Angular*, além de proporcionar clareza, organização e manutenção simplificada.

A seguir, a figura 1 ilustra a relação entre as três camadas da arquitetura.

Figura 1 – Arquitetura MVC



Fonte: Autoria própria

A figura 1 representa o fluxo de comunicação entre as camadas *Model*, *View* e *Controller* do padrão MVC. O *Controller* atua como intermediário entre a interface e a lógica de negócio, recebendo as ações do usuário e direcionando-as ao *Model*, que processa os dados e devolve as informações necessárias para a *View*. Esta, por sua vez, é responsável por apresentar os resultados ao usuário. Essa separação clara de responsabilidades garante maior organização, reutilização de código e facilidade de manutenção da aplicação.

3.1.2 Banco de Dados SQL

Segundo Souza e Boer (2023), um banco de dados pode ser compreendido como uma coleção estruturada de dados inter-relacionados, que representam informações acerca de um domínio específico. Essa estruturação é fundamental para assegurar a coerência, a lógica e a utilidade dos dados, de modo que eles possam atender a objetivos previamente estabelecidos e a demandas reais de sistemas computacionais. Os autores destacam que os principais propósitos de sua utilização são a redução da redundância, a preservação da consistência, a integridade dos dados e a otimização no acesso e na atualização das informações, elementos que garantem maior eficiência e confiabilidade em aplicações corporativas e acadêmicas.

O modelo relacional, introduzido por Codd em 1970, consolidou-se como paradigma dominante no armazenamento de dados. Nesse modelo, as informações são representadas em tabelas, nas quais cada linha corresponde a um registro e cada coluna a um atributo. Essa estrutura favorece a clareza na organização e a flexibilidade na manipulação, além de permitir a definição explícita de relacionamentos entre entidades. Este modelo, portanto, não apenas sistematiza o armazenamento, mas também viabiliza a aplicação de regras de integridade e consistência que asseguram a qualidade dos dados ao longo do tempo.

Uma característica essencial dos bancos de dados relacionais é o uso da *Structured Query Language* (SQL) como linguagem padrão de gerenciamento. Souza e Boer (2023) apontam que a SQL foi reconhecida por organismos internacionais como ANSI e ISO, tornando-se uma linguagem universal para definição, manipulação e consulta de dados. Os comandos SQL são organizados em subconjuntos, cada um responsável por um aspecto específico da interação com o banco de dados, a saber:

DML (*Data Manipulation Language*): comandos de inserção, atualização e exclusão de registros;

DDL (*Data Definition Language*): comandos de definição e gerenciamento da estrutura do banco, como criação e exclusão de tabelas e índices;

DQL (*Data Query Language*): comandos de consulta e filtragem de dados;

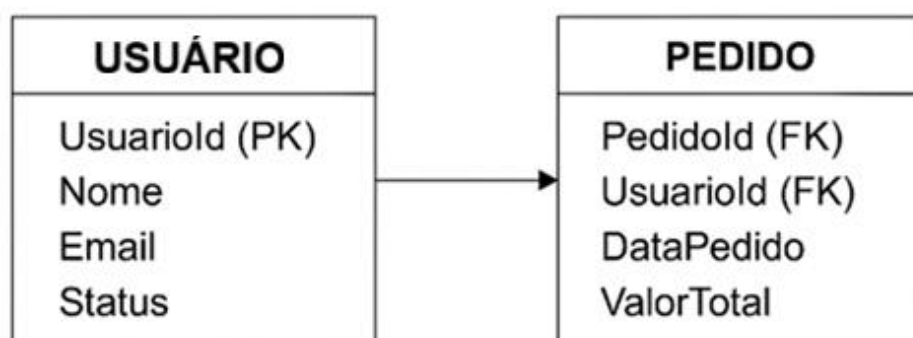
DCL (*Data Control Language*): comandos voltados ao controle de permissões e segurança;

DTL (*Data Transaction Language*): comandos de gerenciamento e controle de transações.

Além da linguagem, Souza e Boer (2023) ressaltam que outro aspecto central do modelo relacional são as propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade), aplicáveis às transações. Essas propriedades asseguram que as operações realizadas sejam executadas de forma íntegra e confiável, mesmo em cenários de falhas ou concorrência de múltiplos acessos simultâneos. A atomicidade garante que uma transação seja executada por completo ou não seja executada; a consistência mantém a validade lógica do banco; o isolamento impede interferências entre transações concorrentes; e a durabilidade assegura que os dados persistam de maneira correta mesmo diante de falhas do sistema.

A Figura 2 apresenta o modelo relacional de dados do sistema, mostrando a relação entre as entidades Usuário e Pedido. A chave primária (Usuarioid) da tabela Usuário é utilizada como chave estrangeira em Pedido, estabelecendo o vínculo entre cliente e pedidos e garantindo a integridade referencial do banco de dados.

Figura 2 – Representação gráfica SQL



Fonte: Autoria própria

O diagrama evidencia como as tabelas se relacionam no banco de dados, permitindo a integração das informações entre clientes e pedidos.

No contexto deste trabalho, a adoção de um banco de dados SQL mostrou-se essencial para o desenvolvimento da aplicação, uma vez que proporcionou uma estrutura sólida para o armazenamento e manipulação das informações, além de oferecer suporte eficiente à comunicação entre o *back-end*, desenvolvido em *ASP.NET Core*, e o *front-end*, implementado em *Angular*. Dessa forma, a utilização do modelo relacional garantiu maior confiabilidade, escalabilidade e clareza no gerenciamento dos dados, fatores fundamentais para a consistência da solução proposta.

3.1.3 Entity Framework

O *Entity Framework* (EF) é um *framework* de mapeamento objeto-relacional (ORM) para a plataforma .NET, que permite a interação entre aplicações e bancos de dados relacionais de forma simplificada e eficiente. De acordo com Oliveira, Bernardino e Paiva (2023), o EF possibilita a execução de operações como consultas, inserções, atualizações e exclusões de registros sem a necessidade de escrever comandos SQL manualmente, garantindo consistência e integridade dos dados.

No contexto do desenvolvimento da plataforma web, o *Entity Framework* foi utilizado na API responsável pelas requisições do sistema, fornecendo uma camada de persistência robusta e de fácil manutenção. A utilização do EF permite que os desenvolvedores manipulem os dados diretamente por meio de objetos da aplicação, promovendo maior produtividade e reduzindo a probabilidade de erros na manipulação do banco de dados.

Uma das principais vantagens do EF é a adoção da abordagem *Code First*, que possibilita criar e gerenciar as tabelas do banco de dados a partir das classes de domínio da aplicação. Essa abordagem traz diversos benefícios: simplifica a modelagem do banco de dados, facilita alterações na estrutura das tabelas conforme a evolução do sistema, e mantém o código-fonte e a estrutura do banco de dados sincronizados. Dessa forma, mudanças em entidades da aplicação podem ser refletidas automaticamente no banco, sem a necessidade de intervenção manual no SQL, tornando o processo de desenvolvimento mais ágil e seguro.

Além disso, o uso do EF contribui significativamente para a escalabilidade da aplicação, permitindo que novas funcionalidades sejam incorporadas sem comprometer a integridade ou o desempenho do banco de dados. A abstração fornecida pelo framework também facilita testes, manutenção e evolução do sistema, tornando-o mais flexível frente às necessidades futuras.

Portanto, o *Entity Framework*, aliado à estratégia *Code First*, se apresenta como uma ferramenta essencial no desenvolvimento do projeto, garantindo uma gestão eficiente dos dados, maior produtividade no desenvolvimento e segurança na manutenção das informações armazenadas no banco de dados.

3.1.4 Angular

O *Angular* é um *framework open-source* mantido pelo Google, amplamente utilizado para o desenvolvimento de aplicações web modernas, especialmente *Single Page Applications* (SPAs), que permitem a navegação fluida entre telas sem a necessidade de recarregar a página. De acordo com Hoffmann, Pinto e Uriarte (2022), o *Angular* foi desenvolvido por Misko Hevery e Adam Abrons em 2009, tendo como objetivo proporcionar o desenvolvimento de aplicações mais objetivas, reutilizáveis e expressivas. Utilizando *TypeScript* como linguagem principal, o framework oferece tipagem estática, orientação a objetos e recursos avançados de desenvolvimento, o que contribui para maior robustez e manutenibilidade do código.

Segundo Pavaneli (2023), o *Angular* destaca-se entre os *frameworks* mais utilizados no desenvolvimento de aplicações web por oferecer uma arquitetura modular, alto desempenho e ampla comunidade de suporte, características que o tornam adequado para aplicações dinâmicas e de grande escala.

Para o desenvolvimento deste projeto, o *Angular* foi adotado como tecnologia principal do *front-end* devido à sua capacidade de modularizar a aplicação em componentes, permitindo a reutilização de elementos, a organização eficiente do código e a escalabilidade futura da aplicação. A utilização do *framework* possibilitou a criação de uma interface intuitiva e de fácil interação com o usuário, contemplando funcionalidades essenciais para o CRM de produtos sustentáveis, como autenticação de usuários, gerenciamento de *leads* e a execução de operações CRUD (*Create, Read, Update, Delete*) no acesso administrativo.

Conforme destacam Hoffmann, Pinto e Uriarte (2022), o *Angular* possui uma sintaxe de *template* baseada em HTML e diretivas próprias, como a *ngFor*, que permite a iteração dinâmica de elementos e a criação de interfaces mais interativas. Além disso, ele permite integração simplificada com serviços de *back-end* por meio do *HttpClient*, possibilitando operações completas de CRUD, essenciais para o gerenciamento de dados da aplicação, como informações de usuários, serviços cadastrados e transações realizadas. Ferramentas complementares como *Angular Material* e *Bootstrap* também foram utilizadas para estilização, construção de formulários e criação de interfaces responsivas, garantindo uma experiência de uso consistente em diferentes dispositivos.

Dessa forma, o *Angular* demonstrou-se fundamental para o desenvolvimento da plataforma web, possibilitando a entrega de uma aplicação escalável, segura e

eficiente, que atende às necessidades tanto dos prestadores de serviço quanto dos usuários finais.

A seguir, a versão do framework que foi utilizada no desenvolvimento do projeto por meio do comando “ng --version”, conforme pode se observar na figura 3:

Figura 3 – Versão *Angular*

```
C:\Users\lucas\OneDrive\Área de Trabalho\TCC\CRM_TCC\CRM_FRONT\crm_tcc>ng --version

Angular CLI
Angular CLI: 8.3.29
Node: 14.21.2
OS: win32 x64
Angular: 19.2.9
... animations, common, compiler, compiler-cli, core, forms
... localize, platform-browser, platform-browser-dynamic, router
```

Fonte: Autoria própria

A definição clara da versão do framework contribui para a padronização do ambiente de desenvolvimento e facilita eventuais evoluções da aplicação.

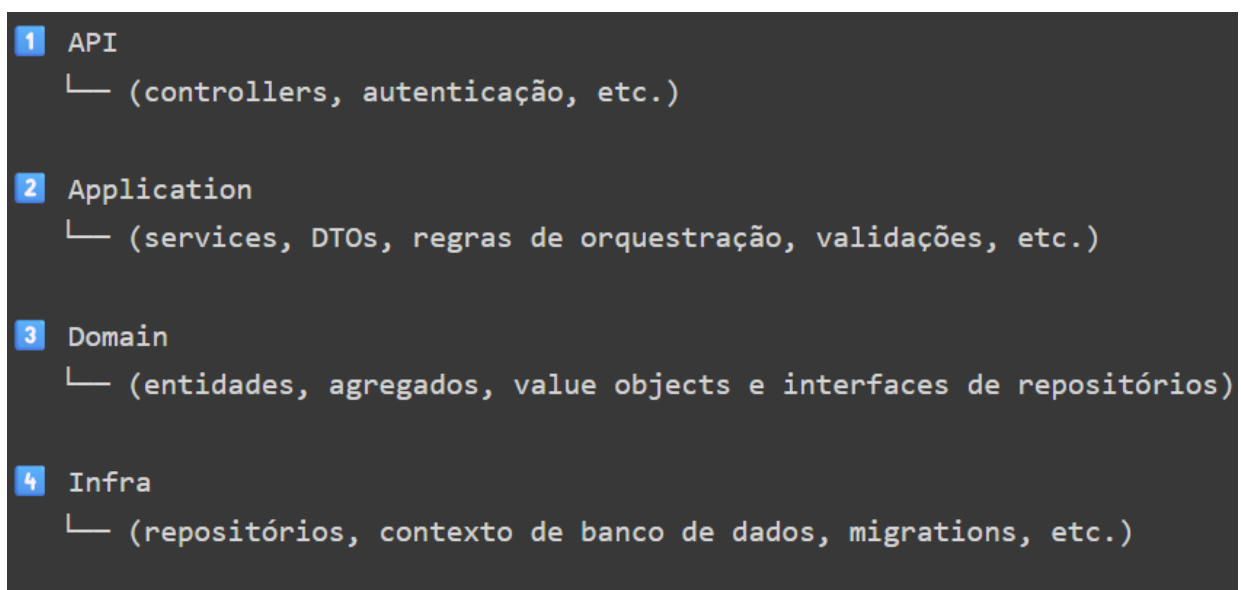
3.2 Processo de desenvolvimento

O processo de desenvolvimento do projeto iniciou-se pela construção do *back-end*, utilizando a IDE *Visual Studio*. Essa etapa foi fundamental, pois consistiu na definição da estrutura da aplicação, configuração do ambiente e escolha das tecnologias que dariam suporte às regras de negócio e à integração com o banco de dados.

3.2.1 Criação e configuração do projeto

O desenvolvimento iniciou-se pelo projeto *back-end*, criado na *IDE Visual Studio*. Para estruturar a solução, foi escolhida a opção *Blank Solution*, permitindo a organização das diferentes camadas de aplicação de forma modular. Após a criação da solução denominada CRM, foram adicionadas as pastas dos projetos correspondentes a cada camada, conforme ilustrado na figura 4 a seguir:

Figura 4 - Arquitetura *back-end*



Fonte: Autoria própria

A estrutura foi organizada em quatro projetos principais, cada um representando uma camada distinta da aplicação:

- **API:** responsável por expor os endpoints do sistema, permitindo a comunicação entre o *front-end* (desenvolvido em *Angular*) e o *back-end*. É nessa camada que estão os *Controllers*, que recebem as requisições, processam os dados e retornam as respostas em formato JSON (*JavaScript Object Notation*), um formato leve de troca de dados baseado em texto, amplamente utilizado por sua simplicidade e compatibilidade entre diferentes linguagens de programação;
- **Application:** concentra a lógica de aplicação, atuando como intermediária entre a camada de domínio e a API. Nela são implementados os casos de uso e as

regras de orquestração, garantindo que as operações sejam executadas de forma consistente;

- *Domain*: representa o núcleo do sistema, onde são definidas as entidades, regras de negócio e contratos. Essa camada é independente de tecnologia e tem como objetivo preservar a lógica essencial do negócio, mantendo o sistema coeso e flexível;
- *Infra*: responsável pela infraestrutura e persistência de dados. Nesta camada foi configurado o *Entity Framework Core*, com os repositórios, *migrations* e implementações de acesso ao banco de dados, além de integrações com serviços externos.

O passo seguinte consistiu na criação dos projetos correspondentes a cada uma destas camadas. Na camada API, foi adicionado o projeto CRM.API, do tipo *ASP.NET Core Web App*. Esse modelo, por padrão, inclui a pasta *Views*, contudo, como o *front-end* foi desenvolvido em um projeto separado, essa pasta foi removida por não ser necessária à estrutura do *back-end*.

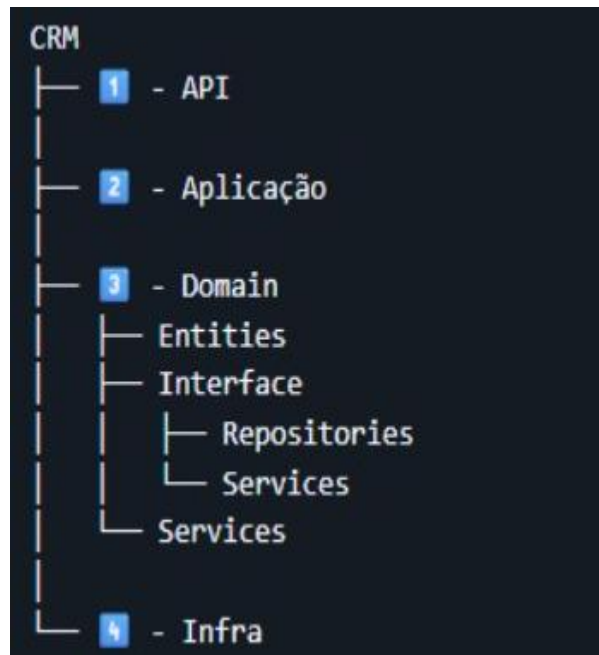
Na camada *Application* (aplicação), o projeto criado foi do tipo *Console App*, servindo como intermediário entre a API e o domínio. Para organizar melhor suas responsabilidades, foram estruturadas três pastas principais: *Interface*, destinada à definição de contratos e serviços; *ModelResponse* (resposta do modelo), responsável por centralizar os objetos de resposta utilizados na comunicação entre as camadas; e *Utils* (utilidades), voltada para classes auxiliares e funcionalidades de apoio ao sistema.

Na camada *Domain* (domínio), também foi utilizado um projeto do tipo *Console App*, cuja finalidade é centralizar as regras de negócio do sistema. Para sua organização, foi estruturado com as seguintes pastas:

- *Entities*: contém as entidades que representam os objetos do negócio, refletindo diretamente os elementos do domínio da aplicação;
- *Interface*: concentra a definição de contratos e foi subdividida em:
 - *Repositories*: responsáveis pela definição das operações de acesso a dados;
 - *Services*: responsáveis pela definição dos serviços relacionados às regras de negócio;
- *Services*: implementa as regras de negócio do sistema, garantindo a aplicação da lógica central do domínio.

Essa organização permite maior modularidade e clareza na separação de responsabilidades, além de manter o domínio independente de aspectos técnicos da aplicação. A figura 5 a seguir mostra como ficou a arquitetura desta camada:

Figura 5 – Camada de Domínio



Fonte: Autoria própria

Por último, criamos o projeto do mesmo modelo das camadas anteriores para a camada de infra, contendo as seguintes subpastas:

- *Context*: responsável pela configuração do contexto de dados, utilizando o *Entity Framework Core* para gerenciar a comunicação com o banco de dados;
- *EntityConfiguration*: armazena as classes de configuração das entidades, definindo mapeamentos, restrições e relacionamentos no modelo de dados;
- *Migrations*: concentra os arquivos de migração gerados pelo *Entity Framework Core*, permitindo a evolução do esquema do banco de dados de forma controlada.
- *Repositories*: contém as implementações concretas dos repositórios, responsáveis pelas operações de persistência e consulta às entidades.

Após a criação e organização das camadas da aplicação, foi necessário configurar os pacotes NuGet responsáveis por fornecer bibliotecas adicionais ao projeto. Esses pacotes possibilitam a implementação de funcionalidades essenciais, como autenticação, persistência de dados e documentação da API. Para adicioná-los, basta ir na opção *Tools* → *NuGet Package Manager* → *Manage NuGet Packages for Solution*.

A figura 6 a seguir mostra os principais pacotes instalados:

Figura 6 - Pacotes NuGet

Pacotes NuGet Utilizados		
 Nome	 Versão	 Descrição
Microsoft.AspNetCore.Authentication.JwtBearer	7.0.18	Middleware para autenticação via JWT
Microsoft.AspNetCore.OData	8.2.5	Suporte a endpoints OData no ASP.NET Core.
Microsoft.AspNetCore.OpenApi	7.0.10	Suporte a anotações e documentação OpenAPI/Swagger
Microsoft.EntityFrameworkCore	7.0.18	ORM para mapeamento de objeto-relacional no .NET.
Microsoft.EntityFrameworkCore.SqlServer	7.0.18	Fornecedor do Entity Framework Core para SQL Server.
Microsoft.EntityFrameworkCore.Tools	7.0.18	Ferramentas para migrações e gerenciamento via EF Core.

Fonte: Autoria própria

Os pacotes instalados e suas respectivas finalidades são apresentados a seguir:

- *Microsoft.AspNetCore.Authentication.JwtBearer*: responsável por habilitar a autenticação baseada em JWT (*JSON Web Token*), permitindo maior segurança no processo de *login* e no controle de acesso aos recursos da API;
- *Microsoft.AspNetCore.OData*: fornece suporte ao *OData* (*Open Data Protocol*), permitindo a criação de *endpoints* mais dinâmicos, com recursos como filtros, paginação e ordenação diretamente nas requisições;
- *Microsoft.AspNetCore.OpenApi*: utilizado para gerar documentação automatizada da API por meio do *Swagger/OpenAPI*, facilitando o teste e a integração dos serviços;
- *Microsoft.EntityFrameworkCore*: biblioteca central do *Entity Framework Core*, responsável pelo mapeamento objeto-relacional (ORM) e pela manipulação de

dados de forma simplificada, através de consultas LINQ (*Language Integrated Query*) e controle de *migrations*;

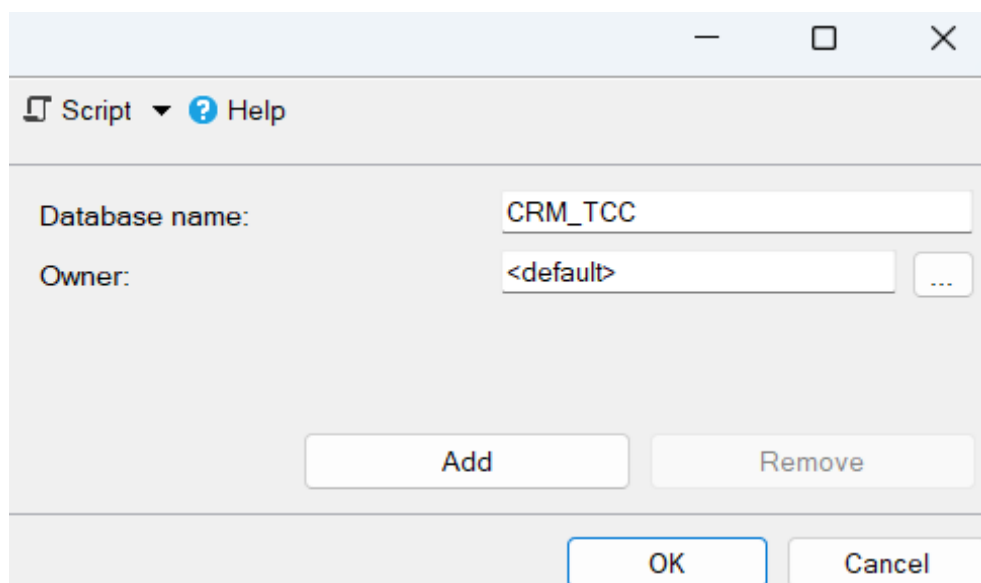
- *Microsoft Entity Framework Core Sql Server*: provedor específico para integração do *Entity Framework Core* com o *Microsoft SQL Server*, banco de dados escolhido para o sistema;
- *Microsoft Entity Framework Core Tools*: fornece ferramentas adicionais para a execução de *migrations* e demais operações de gerenciamento do banco diretamente no *Visual Studio*.

3.2.2 Criação do banco e tabelas SQL

Com a criação das camadas do projeto definidas, a etapa seguinte do desenvolvimento foi a criação do banco de dados seguido da implementação de suas tabelas. Para isso, foi utilizado o SQL Server para criação do banco, e o *Entity Framework* para criação das tabelas via código dentro do projeto, por meio das *migrations*. Este método é conhecido como *Code First*, onde primeiro desenvolvemos o código e depois a criação da tabela em si.

Para criar o banco, uma vez conectado ao SQL, basta clicar com o botão direito do mouse e escolher a opção “*New database*”, no nosso caso o banco foi nomeado como “CRM_TCC”. A figura 7 a seguir mostra a tela de criação do banco:

Figura 7 - Criação do banco SQL



Fonte: Autoria própria

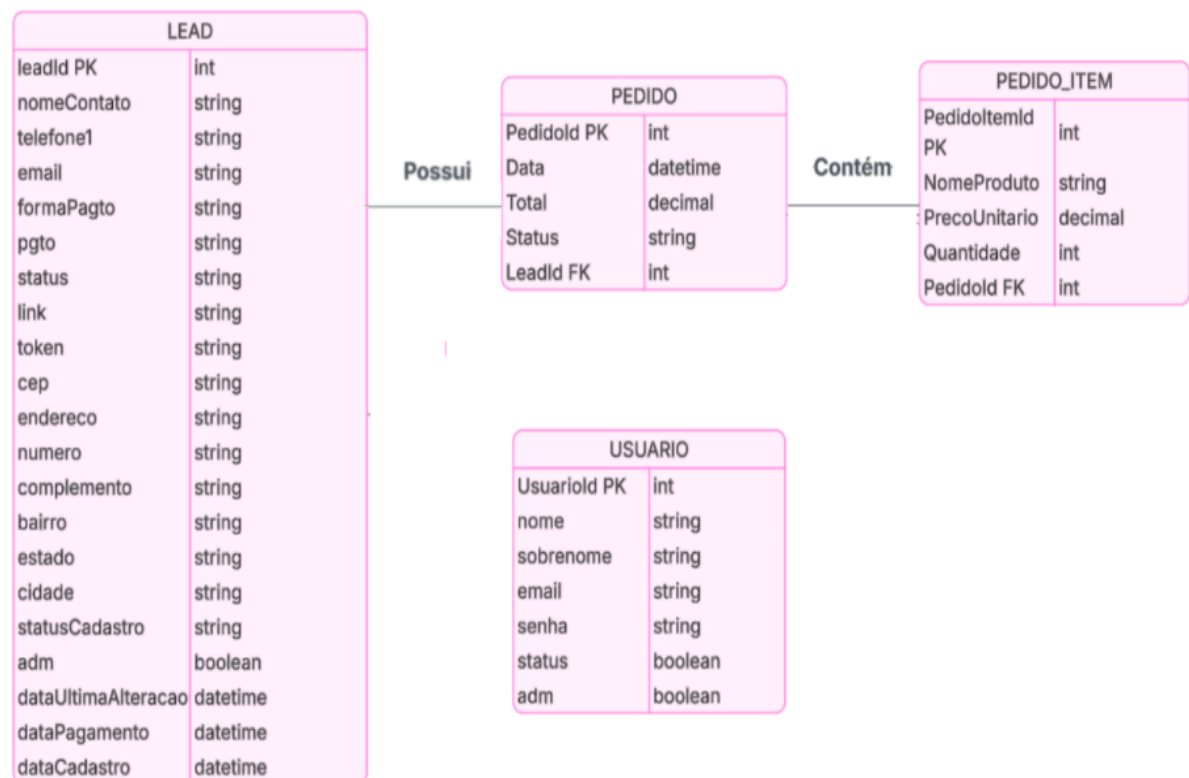
A imagem ilustra o processo de criação do banco de dados CRM_TCC no *SQL Server Management Studio* (SSMS), etapa essencial para estruturar o ambiente de armazenamento das informações utilizadas pelo sistema.

Uma vez criado o banco de dados, estabeleceu-se que o projeto seria estruturado a partir das seguintes tabelas principais:

- Usuário: destinada ao armazenamento das credenciais e informações de acesso administrativo do sistema;
- Lead: armazena dados do cliente;
- Pedido: vinculada à tabela *Lead*, com a finalidade de registrar as informações referentes aos pedidos de compra;
- PedidoItem: associada à tabela *Pedido*, destinada ao armazenamento dos produtos vinculados a cada pedido realizado pelo cliente.

Na sequência, apresenta-se o diagrama que ilustra a estrutura das tabelas e seus respectivos relacionamentos:

Figura 8 - Tabelas do sistema



Para a criação das tabelas, o primeiro passo consistiu na definição da *connection string* (string de conexão), responsável por estabelecer a comunicação entre o projeto e o banco de dados. Nessa configuração, foram especificados o nome do servidor e o nome do banco que o sistema deverá acessar.

A conexão está localizada na camada de API, especificamente no arquivo *appsettings.json*. A seguir, a figura 9 apresenta a configuração utilizada para a conexão com o banco de dados:

Figura 9 – String de conexão

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "con":
    "Server=LUCAS\\SQLEXPRESS;Database=CRM_TCC;Trusted_Connection=True;MultipleActiveResultSets=True;Encrypt=False;TrustServerCertificate=True;"
  }
}
```

Fonte: Autoria própria

A imagem apresenta o conteúdo do arquivo *appsettings.json*, localizado na camada de API do sistema, onde são definidas as configurações essenciais da aplicação. Nesse trecho, observa-se a configuração de log e a *string* de conexão responsável por estabelecer a comunicação com o banco de dados SQL Server, utilizando autenticação integrada e parâmetros que garantem a segurança e a estabilidade da conexão.

Com a conexão estabelecida entre o projeto e o banco, foi criada as entidades das tabelas. Para isso, foram adicionados, na camada de domínio, arquivos de classe destinados a representar cada uma dessas entidades. Em seguida, criado o arquivo de configuração destas entidades na camada de infra do projeto, na pasta *EntityConfigurations*. Por fim, adicionado o arquivo *ContextDb.cs* na pasta *Context*.

Iniciando pela tabela responsável pelo acesso administrativo (Usuário), a classe correspondente foi definida na camada de API, conforme ilustrado na figura 10 a seguir:

Figura 10 – Entidade Usuário

```
public class Usuario
{
    [Key]
    public int Usuariold { get; set; }
    public string nome { get; set; }
    public string sobrenome { get; set; }
    public string email { get; set; }
    public string senha { get; set; }
    public bool status { get; set; }
    public bool adm { get; set; }
}
```

Fonte: Autoria própria

O código apresentado define a classe “Usuario”, que representa a entidade de usuário no sistema. Essa classe contém propriedades que correspondem aos campos da tabela de usuários no banco de dados, como nome, sobrenome, e-mail e senha, além de atributos que indicam o status do usuário e se ele possui privilégios de administrador. O atributo “Key” identifica a propriedade *Usuariold* como a chave primária, utilizada para diferenciar de forma única cada registro.

Após definição da entidade, foi inserido na camada de infra o seguinte código de configuração, conforme figura 11 a seguir:

Figura 11 – Configuração Usuário

```
public class configUsuario : IEntityTypeConfiguration<Usuario>
{
    public void Configure(EntityTypeBuilder<Usuario> builder)
    {
        builder.HasKey(x => x.Usuariold);
        builder.Property(x => x.nome).IsRequired().HasColumnType("varchar(50)");
        builder.Property(x => x.sobrenome).HasColumnType("varchar(200)");
        builder.Property(x => x.email).IsRequired().HasColumnType("varchar(200)");
        builder.Property(x => x.senha).IsRequired().HasColumnType("varchar(200)");
        builder.Property(x => x.status).IsRequired().HasColumnType("bit");
        builder.Property(x => x.adm).HasColumnType("bit");
    }
}
```

Fonte: Autoria própria

O trecho de código define a classe *configUsuario*, responsável por configurar o mapeamento da entidade “Usuario” no *Entity Framework Core*. Por meio da implementação da interface *IEntityTypeConfiguration<Usuario>*, são especificadas as regras de mapeamento entre a classe e a tabela no banco de dados, como a definição da chave primária (*HasKey*) e os tipos de dados das colunas, além de indicar quais propriedades são obrigatórias (*IsRequired*). Essa configuração garante que a estrutura da tabela no banco de dados esteja alinhada com o modelo da aplicação.

As demais entidades (Lead, Pedido e PedidoItem) seguem a mesma estrutura, cada uma com sua respectiva classe de entidade na camada de domínio e arquivo de configuração na pasta *EntityConfigurations*.

Por fim, destaca-se o arquivo *ContextDb.cs*, presente na pasta *Context*, que é responsável por centralizar o mapeamento das entidades e estabelecer a comunicação entre a aplicação e o banco de dados. É por meio dessa classe de contexto que o *Entity Framework* realiza o mapeamento objeto-relacional (ORM), permitindo a manipulação das tabelas a partir das entidades definidas no código.

As figuras 12 e 13 a seguir mostram as partes do arquivo onde são configuradas os mapeamentos e conexão com o banco:

Figura 12 – Mapeamento

```
public class ContextDb : DbContext
{
    public IConfigurationRoot? Configuration { get; set; }
    public DbSet<Usuario> Usuario { get; set; }
    public DbSet<Lead> Lead { get; set; }
    public DbSet<Pedido> Pedido { get; set; }
    public DbSet<PedidoItem> PedidoItem { get; set; }
    public ContextDb()
    {
    }
}
```

Fonte: Autoria própria

A Figura 12 apresenta a classe *ContextDb*, responsável por realizar o mapeamento das entidades do sistema com o banco de dados. Nela são definidos os conjuntos de dados (*DbSet*) correspondentes às principais tabelas da aplicação, permitindo que o *Entity Framework Core* gerencie as operações de persistência de forma automatizada.

Em seguida, os métodos de conexão com o banco e criação das tabelas, conforme a figura 13:

Figura 13 – Método de conexão

```
protected override void OnConfiguring(DbContextOptionsBuilder optionBuilder)
{
    if (!optionBuilder.IsConfigured)
    {
        optionBuilder.UseSqlServer(retornaUrlConexao());
    }
}

public string retornaUrlConexao()
{
    var builder = new
ConfigurationBuilder().SetBasePath(Directory.GetCurrentDirectory()).AddJsonFile("appsettings.json");

    Configuration = builder.Build();

    return Configuration.GetConnectionString("con");
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfiguration(new configUsuario());
    modelBuilder.ApplyConfiguration(new configLead());
    modelBuilder.ApplyConfiguration(new configPedido());
    modelBuilder.ApplyConfiguration(new configPedidoItem());
}
}
```

Fonte: Autoria própria

A figura apresenta os métodos responsáveis pela configuração da conexão com o banco de dados e pela criação das tabelas por meio do *Entity Framework Core*. O método “*OnConfiguring*” define o uso do provedor *SQL Server*, utilizando a *string* de conexão obtida pelo método *retornaUrlConexao()*, que lê o arquivo *appsettings.json*. Já o método “*OnModelCreating*” aplica as configurações de mapeamento das entidades por meio das classes de configuração específicas, garantindo que a estrutura do banco seja criada conforme o modelo definido na aplicação.

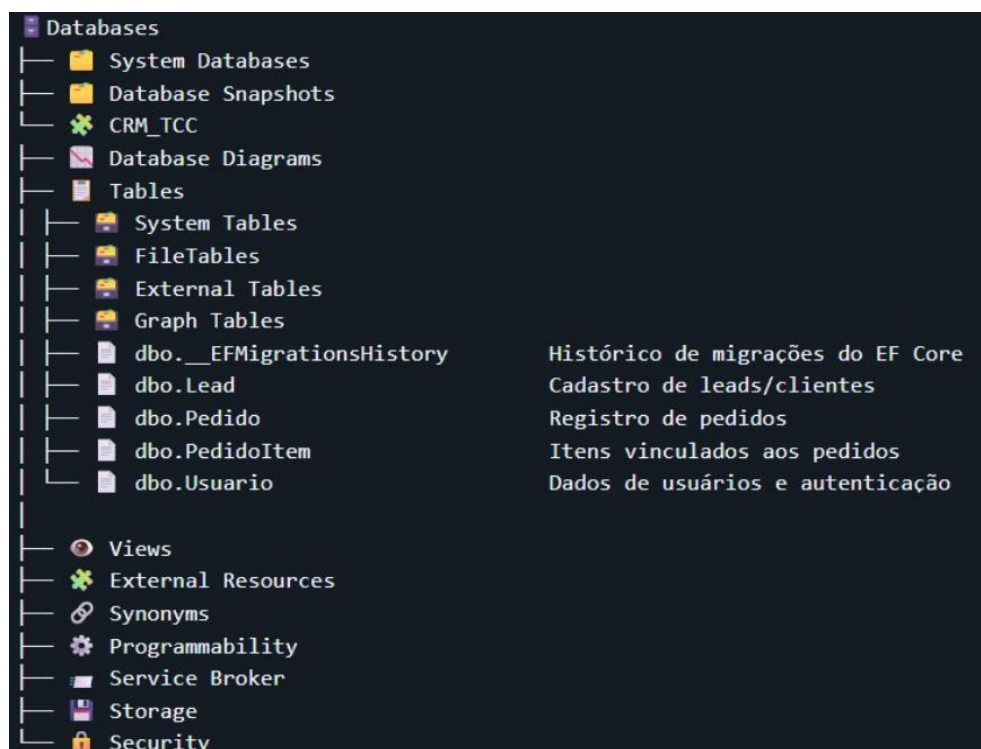
Após a criação das entidades, de seus respectivos arquivos de configuração e da definição do contexto, torna-se necessário gerar as migrações, que são responsáveis por traduzir as classes de domínio e suas configurações em instruções SQL para a criação das tabelas no banco de dados.

No *Entity Framework Core*, esse processo é realizado por meio do comando “Add-Migration”, disponível no *Package Manager Console* do Visual Studio (*Tools* → *NuGet Package Manager* → *Package Manager Console*). É importante ressaltar que esse comando deve ser executado tendo como projeto de destino (*Default Project*) a camada *Infra*, pois é nela que se encontram os arquivos de configuração das entidades e do contexto do banco.

A execução do comando gera um arquivo de migração dentro da pasta *Migrations*, que é criada na camada de *infra*, contendo o script necessário para criar ou atualizar as tabelas de acordo com o modelo definido no código. Esse arquivo pode então ser aplicado ao banco de dados através do comando “*Update-Database*”, efetivando a criação da estrutura física das tabelas.

Após a execução do comando e obtendo sucesso, é possível visualizar as tabelas criadas dentro do banco de dados, conforme a figura 14 a seguir:

Figura 14 - Tabelas SQL



Fonte: Autoria própria

A imagem evidencia a estrutura do banco de dados CRM_TCC no *SQL Server*, mostrando as tabelas criadas após a execução das migrações pelo *Entity Framework*. É possível observar as entidades Lead, Pedido, PedidoItem e Usuario, que compõem o modelo relacional do sistema e sustentam o armazenamento e a manipulação das informações da aplicação.

3.2.3 Criação do projeto *Angular*

A criação do projeto foi realizada por meio da *Angular CLI (Command Line Interface)*, uma ferramenta que auxilia na geração e gerenciamento de aplicações *Angular*. O comando utilizado para inicializar o projeto foi: *ng new nome-do-projeto*.

Durante o processo de criação, foram habilitadas as opções de roteamento (*Angular Routing*) e estilização em CSS, configurando o ambiente de acordo com as necessidades da aplicação. Após a finalização, o próprio CLI executa automaticamente o comando “*npm install*”, responsável pela instalação das dependências do projeto por meio do gerenciador de pacotes *Node Package Manager (npm)*.

Uma vez criado o projeto, realizou-se a organização da estrutura interna dentro do diretório *src/app*, com o objetivo de manter o código limpo, modular e de fácil manutenção. Foram criadas pastas específicas para segmentar os diferentes tipos de recursos e responsabilidades do sistema, conforme descrito a seguir:

- *Components*: Contém os componentes visuais reutilizáveis, como o *menu-top*, *shared* e *sidebar*, responsáveis por elementos comuns da interface e pela padronização visual do sistema;
- *Entities*: Reúne as classes de entidades que representam as estruturas de dados utilizadas no *back-end*, mantendo a consistência entre o *front-end* e o banco de dados.
- *Pages*: Pasta principal que agrupa os módulos funcionais da aplicação, divididos em subpastas como cadastros, *login* e telas de listagem entre outros componentes do sistema;
- *Services*: Contém os serviços responsáveis pela comunicação com o *back-end*, realizando as requisições HTTP às *controllers* da API e centralizando a lógica de integração entre as camadas.

Essa estrutura modular facilita a escalabilidade, legibilidade e manutenção do código, garantindo uma organização consistente e aderente às boas práticas de desenvolvimento em *Angular*. A seguir, serão apresentados alguns dos principais componentes do sistema, com destaque para suas funcionalidades e integração com o *back-end*. Devido à complexidade e ao tamanho do projeto, informações detalhadas sobre todos os componentes e módulos estão disponíveis no Apêndice, bem como no repositório oficial do GitHub do projeto, cujo link pode ser consultado para referência completa.

3.2.3.1 Criação do componente de *login*

Para a implementação da tela de autenticação do sistema, foi criado o componente de *login* utilizando o comando disponibilizado pela *Angular* CLI. O processo foi realizado dentro da pasta *pages*, seguindo o padrão de organização previamente definido para os módulos da aplicação. O comando utilizado foi: *ng generate component login*.

Esse comando é responsável por gerar automaticamente a estrutura básica de um componente *Angular*, composta por três arquivos principais:

- *login.component.html* – responsável pela estrutura visual (*template*) do componente, onde são definidos os elementos da interface, como campos de entrada, botões e mensagens de validação;
- *login.component.css* – responsável pela estilização do componente, aplicando as regras visuais e garantindo a identidade visual consistente com o restante da aplicação;
- *login.component.ts* – arquivo que contém a lógica de funcionamento do componente, desenvolvido em *TypeScript*. Nele são implementadas as funções responsáveis pela captura dos dados de *login*, comunicação com o *back-end* e tratamento de respostas de autenticação.

Após a criação do componente, foi implementado no HTML o formulário responsável por capturar as informações de acesso do usuário previamente cadastrado, especificamente o e-mail e o token de acesso, conforme ilustrado na Figura 15 a seguir:

Figura 15 – HTML *Login*

```
<form autocomplete="off" (ngSubmit)="Login()">
  <div class="form-group">
    <label for="email">E-mail</label>
    <input type="text" name="email" id="email"
      [(ngModel)]="loginEmail" class="form-control"
      placeholder="Digite seu e-mail" />
  </div>

  <div class="form-group" style="margin-top:20px;">
    <label for="Token">Token</label>
    <input type="password" name="Token" id="Token"
      [(ngModel)]="loginToken" class="form-control"
      placeholder="Digite o token" />
  </div>

  <!-- Botão de login -->
  <button class="btn btn-primary btn-block mt-4 btn-add"
type="submit">Entrar</button>
</form>
```

Fonte: Autoria própria

A figura apresenta o código HTML responsável pela construção do formulário de *login* do sistema. Nele, são disponibilizados campos para que o usuário insira seu e-mail e token de acesso, previamente cadastrados na base de dados. O formulário utiliza o recurso de *two-way data binding* do *Angular*, por meio da diretiva `[(ngModel)]`, que mantém a sincronização entre os campos da interface e as variáveis do componente.

Após o preenchimento dos campos e o acionamento do botão “Entrar”, é executado o método `Login()`, que tem como função principal acionar a service responsável pela comunicação com a API do *back-end*. Dependendo da resposta retornada pela *controller*, o método realiza o redirecionamento do usuário para a tela adequada, garantindo o fluxo correto de autenticação no sistema. Caso ocorra um erro durante a validação do acesso, uma mensagem é informada em tela ao usuário. A figura 16 a seguir mostra a estrutura do método:

Figura 16 – Método de *Login*

```
Login() {  
  if (!this.loginEmail || !this.loginToken) {  
    this.toastr.error('Informe o e-mail e o token para entrar.', 'Atenção');  
    return;  
  }  
  
  const payload = {  
    email: this.loginEmail,  
    token: this.loginToken  
  };  
  
  this.leadService.loginLead(payload).subscribe({  
    next: (res: any) => {  
      if (res && res.sucesso) {  
        localStorage.setItem('leadStatus', res.statusCadastro);  
        localStorage.setItem('nome', res.nome);  
        localStorage.setItem('leadId', res.leadId.toString());  
        this.router.navigate(['/lead']);  
      } else {  
        this.toastr.error('Credenciais inválidas.', 'Erro');  
      }  
    },  
    error: err => {  
      this.trataErro.handle(err);  
    }  
  });  
}
```

Fonte: Autoria própria

Inicialmente, o método verifica se os campos de e-mail e token foram preenchidos, exibindo uma mensagem de alerta caso contrário. Em seguida, é criado um objeto *payload* contendo os dados de autenticação, que são enviados ao serviço “*leadService.loginLead()*” para validação no *back-end*. Se a resposta indicar sucesso, as informações do usuário como nome, identificador e status do cadastro são armazenadas no *localStorage*, e o sistema redireciona o usuário para a rota principal do módulo de *leads*. Caso contrário, é exibida uma notificação de erro informando que as credenciais são inválidas.

A *service* realiza uma requisição do tipo *POST* à *controller lead/loginLead* no *back-end*. Uma requisição *POST* é um tipo de chamada HTTP utilizada para enviar dados ao servidor, geralmente com o objetivo de criar ou validar informações. Diferentemente de uma requisição *GET*, que apenas solicita dados, o *POST* envia informações no corpo da requisição, como o e-mail e o token do usuário, permitindo

que o servidor realize a validação das credenciais. A figura 17 abaixo mostra a implementação da *service*:

Figura 17 – Serviço de *login*

```
loginLead(formData: any): Observable<any> {  
  const reqHeader = new HttpHeaders({  
    'access-control-allow-origin': '*',  
    'content-type': 'application/json'  
  });  
  
  return this.http.post<any>(`${this.apiUrl}/lead/LoginLead`, formData, { headers:  
    reqHeader });  
}
```

Fonte: Autoria própria

Esse método realiza uma requisição HTTP do tipo *POST* para a rota “*LoginLead*” da API, enviando os dados de *login* recebidos do formulário. Antes do envio, são configurados os cabeçalhos da requisição, definindo o tipo de conteúdo como *JSON* e permitindo o controle de acesso por origem. A resposta retornada pela API é tratada de forma reativa por meio do *Observable*, possibilitando que o componente de *login* receba o resultado da autenticação e execute as ações correspondentes conforme o retorno do servidor.

Na *controller lead*, as informações recebidas do *front-end* são consultadas no banco de dados. Caso os dados correspondam a um registro existente, o método realiza a validação das credenciais e retorna uma mensagem indicando “*Login* realizado com sucesso”. Caso contrário, é enviada uma mensagem de erro, permitindo que o *front-end* execute o tratamento apropriado e informe o usuário sobre a falha na autenticação.

Uma vez que o usuário é autenticado com sucesso, ele é redirecionado para a tela de produtos, que constitui a interface principal do CRM operacional. Nessa tela, o usuário pode visualizar os produtos disponíveis, selecionar os itens desejados e montar seu pedido, seguindo o fluxo de operação do sistema.

3.2.3.2 Montagem do pedido

A tela de montagem de pedido é a interface principal onde o usuário pode visualizar os produtos disponíveis e selecionar os itens que deseja incluir em seu pedido. A listagem de produtos é exibida de forma organizada em cards, apresentando informações como nome, volume, imagem e preço de cada item. Para adicionar produtos ao pedido, o usuário utiliza o botão Adicionar, que inclui o item no carrinho de compras. A figura 18 apresenta o trecho do HTML responsável pela exibição da listagem de produtos:

Figura 18 - Lista de produtos HTML

```
<!-- PRODUTOS (esquerda) -->
<div class="col-md-8">
  <div class="row" style="margin-top:39px;">
    <div class="col-md-4 mb-4" *ngFor="let product of produtosFiltrados">
      <div class="card h-100">
        <img [src]="product.imageUrl" class="card-img-top" alt="{{ product.name
}}">
        <div class="card-body d-flex flex-column">
          <h5 class="card-title">{{ product.name }}</h5>
          <p class="card-text">{{ product.volume }}</p>
          <strong class="mb-2">R$ {{ product.price.toFixed(2) }}</strong>
          <button class="btn btn-adicionar mt-auto btn-sm"
(click)="addToCart(product)">Adicionar</button>
        </div>
      </div>
    </div>
  </div>
</div>
```

Fonte: Autoria própria

Cada item é renderizado dinamicamente por meio da diretiva **ngFor*, que percorre o *array* “produtosFiltrados” e exibe as informações correspondentes a cada produto, como imagem, nome, volume e preço. Além disso, cada card contém um botão “Adicionar”, que aciona o método “*addToCart(product)*” para incluir o produto selecionado no carrinho de compras. Essa estrutura proporciona uma interface organizada e intuitiva, facilitando a visualização e a interação do usuário com os produtos disponíveis.

O carrinho de compras é exibido ao lado da lista de produtos, mostrando todos os itens selecionados pelo usuário, suas quantidades e o total do pedido. O usuário pode aumentar ou diminuir a quantidade, bem como remover produtos do carrinho, garantindo total controle sobre o pedido antes de enviá-lo. Quando o carrinho está vazio, uma mensagem informativa é exibida para o usuário informando que seu pedido no momento não contém nenhum produto. A figura 19 ilustra o HTML responsável pelo carrinho e suas funcionalidades:

Figura 19 - Carrinho de compras

```
<!-- CARRINHO (direita) -->
<div class="col-md-4">
  <strong class="mb-3" style="color: #616D7C;">
    <i class="fas fa-shopping-cart"></i> Meus Produtos
  </strong>

  <div *ngIf="cart.length === 0" class="alert alert-info">Seu carrinho está
vazio.</div>

  <div *ngIf="cart.length > 0" class="card shadow-sm" style="border-
radius:0px;margin-top:15px;">
    <ul class="list-group list-group-flush">
      <li class="list-group-item" *ngFor="let item of cart; let i = index">
        <div class="d-flex justify-content-between align-items-center">
          <div class="d-flex align-items-center">
            <!-- Botão diminuir -->
            <button class="btn btn-sm btn-danger me-2"
(click)="diminuirQuantidade(i)" aria-label="Diminuir quantidade">-</button>

            <!-- Quantidade -->
            <small style="min-width: 25px; text-align: center;">{{ item.quantity
}}</small>

            <!-- Botão aumentar -->
            <button class="btn btn-sm btn-primary-plus ms-2 me-3"
(click)="aumentarQuantidade(i)" aria-label="Aumentar quantidade">+</button>

            <!-- Nome do produto -->
            <strong style="font-size: 14px; margin-left: 10px; color:grey;">{{
item.product.name }}</strong>
          </div>
          <div style="margin-right:3px;">
            <strong style="color:grey;">R$ {{ (item.product.price *
item.quantity).toFixed(2) }}</strong>
          </div>
        </div>
      </li>
    </ul>
  </div>
</div>
```



```

<!-- Botão remover produto -->
<button class="btn btn-sm btn-danger ms-3" (click)="remover(i)" aria-
    label="Remover produto">
    <i class="fas fa-trash"></i>
</button>
</div>
</div>
</li>
</ul>
<div class="card-footer text-end">
    <strong style="color: #616D7C;">Total: R$ {{ getCartTotal() }}</strong>
</div>

```

Fonte: Autoria própria

O componente realiza a renderização condicional do estado vazio e, quando há itens, lista cada produto com quantidade, subtotal e ações para aumentar/diminuir a quantidade ou remover o item. Ao final, é exibido o total do carrinho, calculado dinamicamente a partir dos valores e quantidades selecionados. Essa estrutura oferece ao usuário um controle direto e intuitivo sobre os itens adicionados.

Após selecionar os produtos desejados, o usuário utiliza o botão “Prosseguir para pagamento” para enviar os itens do carrinho e montar o pedido. Esse botão permanece desabilitado enquanto o carrinho estiver vazio e, quando acionado, dispara o método `enviarPedido()`, que organiza os itens selecionados e prepara o pedido para a próxima etapa do fluxo de compra. A figura 20 apresenta o código HTML referente ao botão de envio do pedido:

Figura 20 – Envio do pedido HTML

```

<!-- Botão para enviar o pedido -->
<div class="d-flex justify-content-end mt-3">
    <button class="btn btn-primary-plus" [disabled]="cart.length === 0"
(click)="enviarPedido()">
        Prosseguir para pagamento
    </button>
</div>

```

Fonte: Autoria própria

Conforme a imagem, o botão permanece desabilitado enquanto o carrinho estiver vazio e, ao ser acionado, executa o método “`enviarPedido()`”, responsável por reunir os itens selecionados e iniciar o processo de envio do pedido.

A lógica de montagem e envio do pedido é implementada no arquivo *TypeScript*, que organiza os produtos selecionados pelo usuário, calcula o total e envia os dados para o *back-end* por meio de sua *service*.

O método `montarPedido()` é responsável por criar um objeto do tipo `Pedido`, populando suas informações principais, como o ID do pedido, dados do usuário (*lead*), data, itens do carrinho e total do pedido. Cada item do carrinho é convertido em um objeto `PedidoItem`, contendo nome do produto, quantidade e preço unitário. Ao final, o método retorna o objeto completo, pronto para ser enviado à API. A figura 21 apresenta o código do método:

Figura 21 - Montagem do pedido

```
montarPedido(): Pedido {
  const pedido = new Pedido();

  pedido.pedidoId = this.pedido?.pedidoId ?? 0;
  pedido.lead = this.lead;
  pedido.leadId = this.lead.leadId;

  pedido.data = new Date().toLocaleString("sv-SE");
  pedido.itens = this.cart.map(item => {
    const pedidoItem = new PedidoItem();
    pedidoItem.pedidoItemId = item.pedidoItemId ?? 0;
    pedidoItem.pedidoId = pedido.pedidoId;

    pedidoItem.nomeProduto = item.product.name;
    pedidoItem.quantidade = item.quantity;
    pedidoItem.precoUnitario = item.product.price;
    return pedidoItem;
  });

  pedido.total = this.cart.reduce((acc, item) => acc + (item.product.price *
item.quantity), 0);

  return pedido;
}
```

Fonte: Autoria própria

Nele método são definidos os dados principais, como identificador, data, itens e valor total. Cada produto selecionado é transformado em um objeto `PedidoItem`, contendo nome, quantidade e preço unitário, resultando em um pedido completo e pronto para envio à API.

O método `enviarPedido()` utiliza o resultado de `montarPedido()` para realizar a comunicação com o servidor, enviando uma requisição *POST* contendo os dados do pedido. Dependendo da resposta, o método executa ações apropriadas no *front-end*, como avançar para a etapa de pagamento (*currentStep* = 3) ou exibir mensagens de erro na tela. A figura 22 apresenta o código do método `enviarPedido()`:

Figura 22 - Envio do pedido

```
enviarPedido() {
  const pedido = this.montarPedido();

  this.pedidoService.adicionarPedido(pedido).subscribe({
    next: (res) => {
      if (res.success) {
        this.currentStep = 3;

        setTimeout(() => {
          this.initStripe();
        }, 50);

      } else {
        this.toastr.error(res.message || 'Erro ao enviar pedido');
      }
    },
    error: (err) => {
      this.toastr.error('Erro ao salvar pedido. ');
      console.error(err);
    }
  });
}
```

Fonte: Autoria própria

A comunicação é feita por meio de uma requisição *POST* e, conforme a resposta, o sistema avança para a etapa de pagamento ou exibe mensagens de erro em caso de falha. Esse processo garante a integração entre o *front-end* e a API na finalização da compra.

A *service* de pedidos, por sua vez, é responsável por efetivar a chamada HTTP para a API do *back-end*, adicionando o cabeçalho de autorização e enviando o objeto pedido para a rota: `/pedido/SalvarPedido`. A figura 23 a seguir apresenta o código da *service*:

Figura 23 - Salvar pedido

```
public adicionarPedido(pedido: Pedido): Observable<response> {  
    const reqHeader = new HttpHeaders({  
        'Authorization': 'Bearer ' + localStorage.getItem("token")  
    });  
  
    return this.http.post<response>(`${this.apiUrl}/pedido/SalvarPedido`, pedido, {  
        headers: reqHeader  
    });  
}
```

Fonte: Autoria própria

Este método é responsável por enviar o objeto Pedido ao *back-end* por meio de uma requisição HTTP *POST*. O método adiciona o cabeçalho de autorização com o *token* do usuário e encaminha os dados para a rota */pedido/SalvarPedido*, permitindo que o servidor registre o pedido na base de dados.

Dessa forma, o *front-end* realiza todo o fluxo de montagem e envio do pedido, garantindo que os itens selecionados pelo usuário sejam organizados corretamente e enviados para processamento no *back-end*, mantendo a consistência e integridade das informações.

3.2.3.3 Pagamento

A tela de pagamento foi implementada para permitir que o usuário finalize seu pedido utilizando cartão de crédito, simulando transações por meio da API Stripe, uma plataforma amplamente utilizada no mercado para pagamentos online, reconhecida pela segurança e padronização em suas integrações. Sua implementação por meio do Stripe Elements facilita a captura dos dados do cartão, garantindo conformidade com boas práticas modernas de segurança.

A interface é dividida em duas colunas principais:

- Esquerda: exibe os campos do cartão, incluindo número, validade e CVV, e os botões Pagar e Voltar. Os elementos do cartão são gerenciados pelo *Stripe Elements*, garantindo segurança e padronização na captura das informações.
- Direita: apresenta os produtos selecionados no carrinho, suas quantidades e o total do pedido. Essa seção permite que o usuário revise seu pedido antes de prosseguir com o pagamento.

A figura 24 apresenta o código HTML que implementa essa interface.

Figura 24 - Interface de pagamento

```
<div class="row" style="margin-top:15px;">
  <!-- Coluna esquerda: Pagamento -->
  <div class="col-md-6 border-end pe-4">
    <h5>Pagamento</h5>

    <!-- Stripe Elements: cartão -->
    <div class="mb-3" style="margin-top:15px;">
      <label class="form-label">Número do Cartão</label>
      <div id="card-number" class="p-3 border rounded mb-2" style="background-
color: #f9f9f9;"></div>

      <label class="form-label">Validade (MM/AA)</label>
      <div id="card-expiry" class="p-3 border rounded mb-2" style="background-
color: #f9f9f9;"></div>

      <label class="form-label">CVV</label>
      <div id="card-cvc" class="p-3 border rounded mb-2" style="background-color:
#f9f9f9;"></div>

      <div id="card-errors" role="alert" class="text-danger mt-2"></div>
    </div>

    <button class="btn btn-primary mt-3" (click)="pagar()">Pagar</button>
    <button type="button" class="btn btn-secondary mt-3 ms-2"
(click)="voltar()">Voltar</button>
  </div>

  <!-- Coluna direita: Resumo do pedido -->
  <div class="col-md-6 ps-4">
    <h5>Resumo do Pedido</h5>
    <ul class="list-group list-group-flush">
      <li class="list-group-item d-flex justify-content-between" *ngFor="let item of
cart">
        {{ item.product.name }} x {{ item.quantity }}
        <span>R$ {{ (item.product.price * item.quantity) | number:'1.2-2' }}</span>
      </li>
      <li class="list-group-item d-flex justify-content-between">
        <strong>Total</strong>
        <strong>R$ {{ getCartTotal() | number:'1.2-2' }}</strong>
      </li>
    </ul>
  </div>
</div>
```

Fonte: Autoria própria

O método `pagar()`, implementado no arquivo *TypeScript*, é responsável por processar o pagamento do pedido. Ele realiza os seguintes passos principais:

- Validação inicial: verifica se o pedido existe e se os elementos do *Stripe* foram inicializados corretamente;
- Criação da intenção de pagamento: chama a *service* de pedidos para criar a intenção de pagamento no *back-end*, recebendo o *clientSecret* necessário para confirmar a transação;
- Confirmação do pagamento: utiliza o *stripe.confirmCardPayment* com os dados do cartão para efetivar o pagamento;
- Tratamento de erros: exibe mensagens de erro na tela caso haja falha na transação ou na inicialização do *Stripe*;
- Atualização do sistema: em caso de sucesso, o método marca o pagamento como realizado, atualiza o status do lead para “Finalizado”, registra a forma de pagamento e envia as informações para o *back-end*, garantindo consistência dos dados;
- Navegação: redireciona o usuário para a tela de confirmação do pedido finalizado (`/lead-finalizado`).

As figuras a seguir apresentam o código completo do método `pagar()`, dividido em partes para melhor visualização:

Figura 25 – Início do pagamento

```
async pagar() {
  if (!this.pedido) {
    this.toastr.error('Pedido não encontrado!');
    return;
  }
  const valor = this.getCartTotal();

  // Verifica se o Stripe e os elementos do cartão já foram inicializados
  if (!this.stripe || !this.cardElement || !this.cardElement.cardNumber) {
    this.toastr.error('Stripe não inicializado.');
```

```
    return;
  }
  try {
    // Cria intenção de pagamento no backend
    const res: any = await this.pedidoService.criarPagamento(valor).toPromise();
    if (!res.clientSecret) {
      this.toastr.error('Não foi possível iniciar o pagamento.');
```

```

    return;
  }
  // Confirma o pagamento usando apenas o cardNumber
  const { error } = await this.stripe.confirmCardPayment(res.clientSecret, {
    payment_method: {
      card: this.cardElement.cardNumber
    }
  });

  const errorDiv = document.getElementById('card-errors');

```

Fonte: Autoria própria

O primeiro trecho do método realiza a validação inicial do pedido e dos elementos do *Stripe*. Caso o pedido não exista ou os objetos necessários para o pagamento não estejam inicializados, o sistema exibe mensagens de erro e interrompe o processo. Essa etapa é essencial para evitar transações inconsistentes e garantir que o pagamento só prossiga quando todos os dados estiverem devidamente preparados. Em seguida, é feita a criação da intenção de pagamento no *back-end*, por meio do serviço *pedidoService*, que retorna o *clientSecret* usado na autenticação da operação.

A figura 26 a seguir, mostra a continuação do método:

Figura 26 – Atualização do lead

```

if (error) {
  if (errorDiv) errorDiv.textContent = error.message ?? "";
  this.toastr.error(error.message || 'Erro no pagamento.');
```

```

} else {
  if (errorDiv) errorDiv.textContent = "";
  this.toastr.success('Pagamento realizado com sucesso!');
  this.router.navigateByUrl('/lead-finalizado');
```

```

  if (!this.pedido?.lead) {
    this.toastr.error('Lead não encontrado no pedido.');
```

```

    return;
  }

  this.lead = this.pedido.lead;
  this.lead.pgto = true;
  this.lead.statusCadastro = 'Finalizado';
  this.lead.formaPagto = 'Cartão';
  this.lead.dataPagamento = new Date().toISOString();
  this.currentStep = 3;

  // Atualiza lead no backend
  this.leadService.updateLead(this.lead).subscribe({

```

```

next: (res) => {
  if (!res.success) {
    this.toastr.error(res.message || 'Erro ao atualizar status do lead.');
```

Fonte: Autoria própria

A segunda parte do método é responsável por confirmar o pagamento. Caso a operação seja bem-sucedida, o sistema apresenta uma notificação de sucesso e redireciona o usuário para a página de confirmação do pedido. Além disso, o código atualiza o objeto *lead*, marcando o pagamento como concluído, definindo o status como “Finalizado” e registrando a data e a forma de pagamento. Essa etapa garante a sincronia entre as informações do pagamento e os dados do cliente armazenados no sistema. Em seguida, a figura 27 mostra a o trecho final do método:

Figura 27 – Atualização do pedido

```

//Atualiza Pedido
if (!this.pedido || this.pedido.pedidoId === undefined) {
  this.toastr.error('Pedido não encontrado!');
  return;
}
this.pedidoService.atualizaStatus(this.pedido.pedidoId, 'Pago').subscribe({
  next: res => {
    if (!res.success) {
      this.toastr.error(res.message || 'Erro ao atualizar status do pedido.');
```

Fonte: Autoria própria

Por fim, a última parte do método executa a atualização do status do pedido no *back-end*, marcando-o como “Pago”. Essa atualização é feita por meio da função “pedidoService.atualizaStatus”, que garante que o banco de dados reflita corretamente o estado final da transação. Caso ocorra algum erro durante essa atualização, o sistema exibe mensagens informativas para o usuário, facilitando o processo de depuração e controle. Essa etapa encerra o fluxo de pagamento, consolidando a transação e garantindo a consistência entre o *front-end* e o servidor.

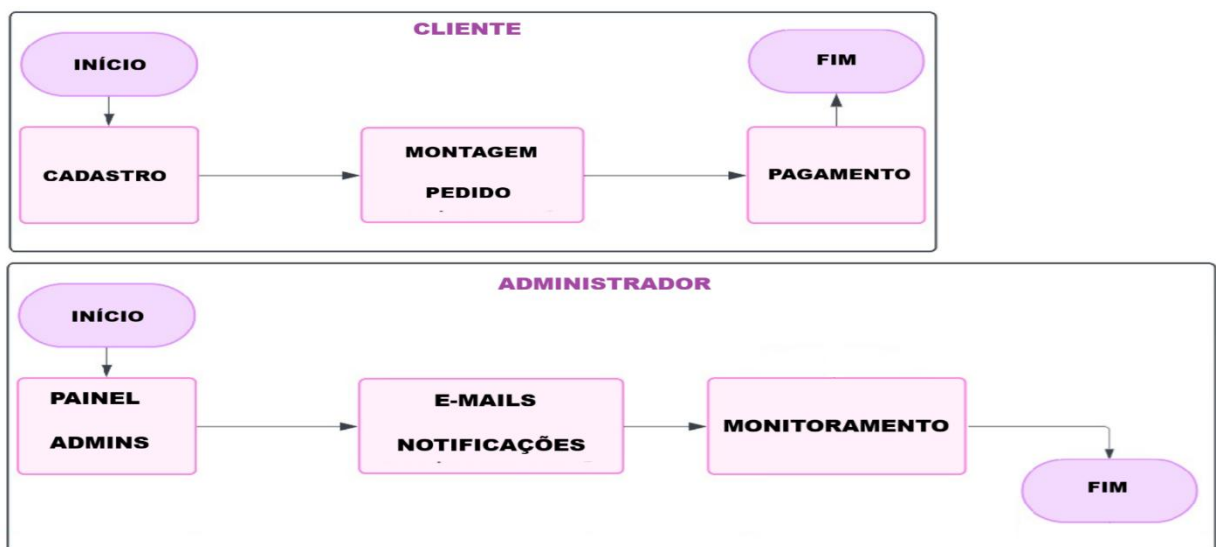
O fluxo garante que o pagamento seja processado corretamente e que o status do pedido e do lead seja atualizado no sistema, concluindo o processo de compra no CRM operacional.

4 RESULTADOS OBTIDOS

Após a conclusão do desenvolvimento, foi possível obter um sistema CRM Operacional funcional, voltado à gestão de clientes e pedidos no contexto de empresas de produtos sustentáveis. O sistema apresenta dois fluxos principais de navegação, que refletem as diferentes perspectivas de uso dentro da aplicação. O primeiro corresponde à jornada do cliente, desde o momento em que realiza seu cadastro até a finalização do pedido e confirmação do pagamento. Já o segundo fluxo representa a visão gerencial, na qual o administrador do sistema pode acompanhar o status dos clientes, receber notificações automáticas e monitorar as etapas do processo comercial em tempo real.

A figura 28 a seguir mostra o fluxograma de navegação de ambas as partes:

Figura 28 - Fluxograma de navegação



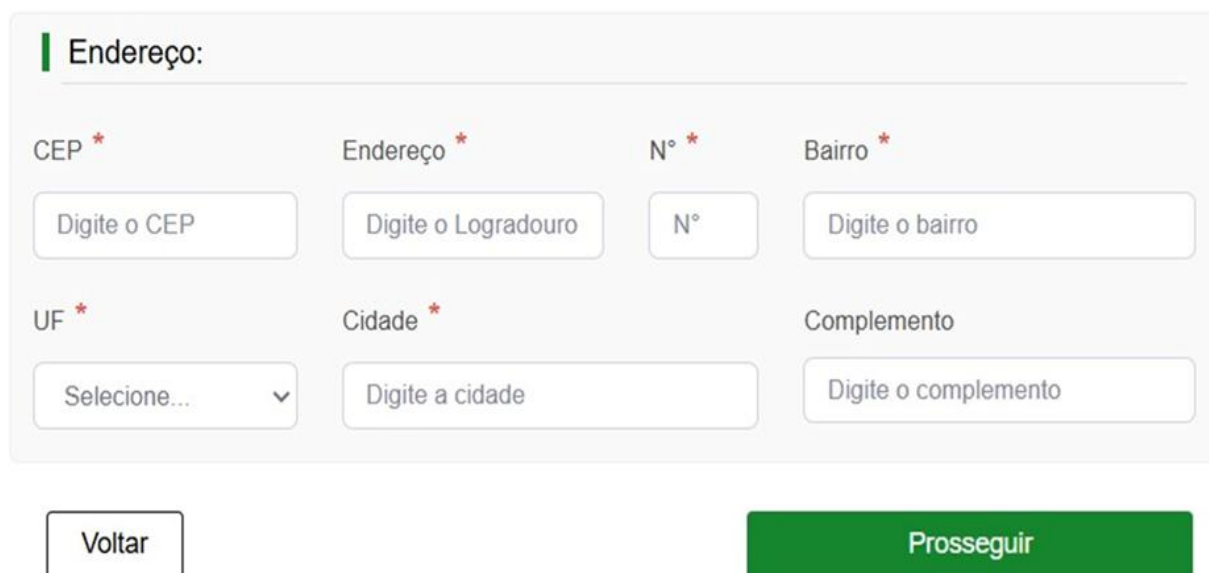
Fonte: Autoria própria

4.1 Fluxo Cliente

No fluxo do cliente, o usuário inicia o processo acessando a plataforma e selecionando a opção de cadastro. Inicialmente, são solicitadas informações básicas de usuário, como nome e e-mail, garantindo a criação de uma conta única e segura. Em seguida, o sistema direciona o cliente para o preenchimento dos dados de endereço. Todo esse processo foi projetado para ser intuitivo e ágil, de forma a proporcionar uma experiência satisfatória desde o primeiro contato com a plataforma.

Para melhor compreensão do processo, a figura 29 abaixo apresenta parcialmente os campos de preenchimento exigidos no cadastro de usuário:

Figura 29 - Cadastro



O formulário, intitulado "Endereço:", contém campos para CEP, Endereço, N°, Bairro, UF, Cidade e Complemento. Os campos CEP, Endereço, N°, Bairro, UF e Cidade são obrigatórios, indicados por um asterisco vermelho. O campo Complemento é opcional. Abaixo dos campos, há dois botões: "Voltar" e "Prosseguir".

CEP *	Endereço *	N° *	Bairro *
Digite o CEP	Digite o Logradouro	N°	Digite o bairro

UF *	Cidade *	Complemento
Selecione...	Digite a cidade	Digite o complemento

Fonte: Autoria própria

Para melhor compreensão do processo, a figura acima apresenta a tela destinada ao preenchimento de endereço do usuário. Nela, é possível observar os campos obrigatórios de CEP, Endereço, Número, Bairro, UF e Cidade, além do campo opcional de Complemento. A interface ainda disponibiliza os botões “Voltar” e “Prosseguir”, que permitem navegar entre as etapas do cadastro, garantindo um fluxo intuitivo e organizado para o usuário.

Após a conclusão do cadastro, o usuário é redirecionado para a tela de *login*, momento em que o sistema informa que, em breve, ele receberá um e-mail contendo o token de acesso para autenticação na plataforma. Esse procedimento garante a segurança do acesso e validação da conta, assegurando que apenas usuários cadastrados possam utilizar o sistema. A figura 30 a seguir apresenta o modelo do e-mail enviado automaticamente aos usuários recém-cadastrados:

Figura 30 - E-mail de boas vindas



Fonte: Autoria própria

A mensagem no corpo do e-mail confirma o sucesso do cadastro e fornece os dados de acesso, como o e-mail registrado e o *token* de autenticação, além de um botão que direciona o usuário para a plataforma. Esse e-mail tem a função de validar o cadastro e facilitar o primeiro acesso ao sistema.

Após o *login*, o usuário é direcionado para a tela de produtos, onde tem a possibilidade de montar seu pedido de forma prática e personalizada. Nesta etapa, ele pode adicionar produtos ao carrinho de compras, bem como remover itens conforme desejar, garantindo flexibilidade na composição do pedido. Além disso, o sistema disponibiliza ferramentas de busca e filtragem, permitindo ao usuário organizar os produtos por diferentes critérios, como preço crescente ou decrescente, ordem alfabética ou localizar diretamente um item específico por meio do campo de pesquisa. Essas funcionalidades foram implementadas com o objetivo de proporcionar uma experiência de compra mais ágil, intuitiva e satisfatória, facilitando a navegação e a escolha dos produtos disponíveis na plataforma. A figura 31 ilustra a tela de produtos, evidenciando as opções de seleção, adição e filtragem de itens:

Figura 31 - Produtos

Buscar

Nome (A to Z)



Planta Decorativa

1Kg

R\$ 175.00

Adicionar



Lava louças

200ml

R\$ 180.00

Adicionar



Not Milk

1 litro

R\$ 14.69

Adicionar



Meus Produtos

-

1

+

Lixeira Inox Sustentável

R\$ 180.00



Total: R\$ 180

Prosseguir para pagamento

Fonte: Autoria própria

Após a montagem do pedido, todas as informações são registradas e armazenadas no banco de dados, vinculadas à conta do usuário, garantindo que cada transação esteja corretamente associada ao cliente responsável. Concluída essa etapa, o usuário é automaticamente direcionado para a fase de pagamento, na qual, por meio da API de pagamento *Stripe*, é realizada a simulação da compra. Esse processo permite validar o fluxo de transação, assegurando que os dados do pedido sejam processados de forma segura e consistente, além de proporcionar uma experiência realista de finalização de compra, mesmo em ambiente de teste ou protótipo. Na figura 32 é possível observar a tela de pagamento, evidenciando o fluxo de conclusão da compra com cartão de crédito:

Figura 32 - Pagamento

Pagamento

Número do Cartão

1234 1234 1234 1234

Utilizar link

Validade (MM/AA)

MM/AA

CW

CVC

Pagar

Voltar

Resumo do Pedido

Kit copos x 1	R\$ 135.00
Total	R\$ 135.00

Fonte: Autoria própria

Após a confirmação do pagamento, o sistema exibe uma tela final informando que a compra foi concluída com sucesso. Nessa etapa, o usuário dispõe de duas opções: a primeira permite continuar comprando, sendo redirecionado novamente para a tela de produtos para adicionar novos itens ao carrinho. A segunda, oferece a possibilidade de encerrar a sessão, levando o usuário de volta à tela de *login*. Essa abordagem proporciona flexibilidade e controle sobre a navegação, garantindo uma experiência de uso organizada.

A figura 33 apresenta a tela de confirmação de compra, evidenciando as opções disponíveis ao usuário: continuar comprando ou encerrar a sessão e retornar à tela de *login*.

Figura 33 - Compra Finalizada



Fonte: Autoria própria

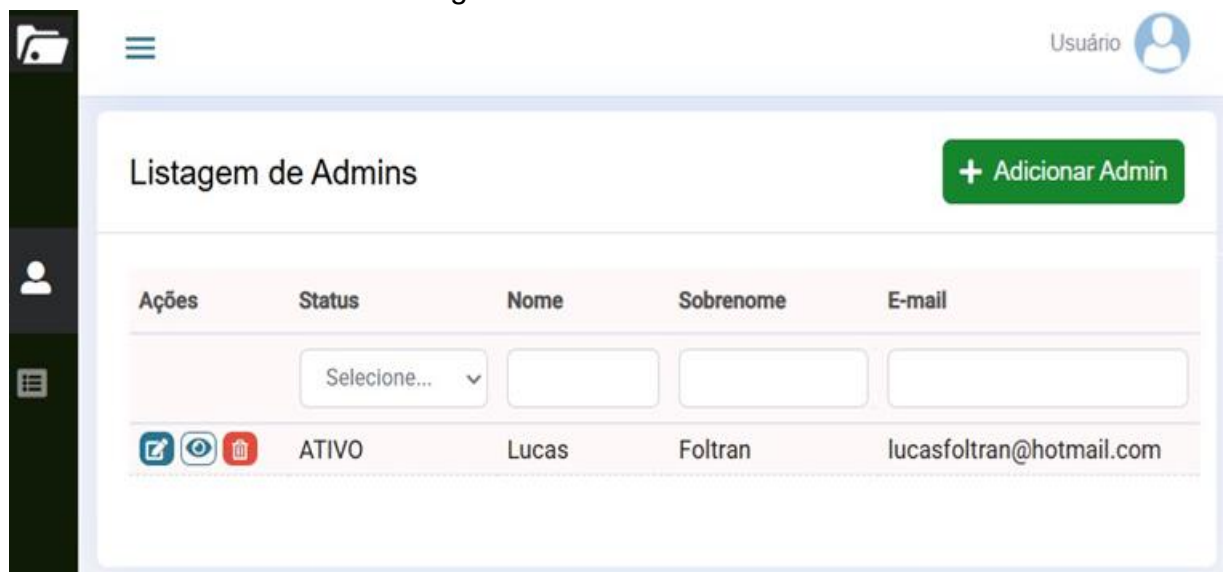
A figura acima apresenta a tela de confirmação de compra do sistema EcoStore, exibida ao término do processo de pagamento.

4.2 Fluxo Administrativo

Ao acessar o sistema, o administrador dispõe de um painel de controle que apresenta informações sobre os usuários cadastrados e outros administradores. No caso dos usuários, a visualização se limita aos campos nome, e-mail e status da compra, permitindo acompanhar em que etapa cada cliente se encontra, desde o cadastro até a confirmação do pagamento. Além disso, o administrador possui

funcionalidades de gerenciamento de outros administradores, incluindo a opção de adicionar novos perfis, garantindo controle sobre quem possui acesso às ferramentas de gestão do sistema e permitindo a manutenção de uma estrutura de administração segura e organizada. A figura 34 apresenta o painel de gerenciamento de administradores, evidenciando as funcionalidades disponíveis para visualização de usuários, acompanhamento do status das compras e adição de novos perfis de administrador:

Figura 34 - Acesso Admin



Fonte: Autoria própria

A figura apresenta a interface de listagem de administradores do sistema EcoStore, na qual é possível visualizar informações como nome, sobrenome, e-mail e status de cada administrador, além das opções de edição, exclusão e adição de novos perfis.

Sempre que um usuário realiza seu cadastro no sistema, é acionado automaticamente o envio de notificações por e-mail para todos os administradores cadastrados, contendo as informações fornecidas pelo cliente, como nome, e-mail e dados de endereço. Esse procedimento tem um papel fundamental no fluxo administrativo, pois garante que os administradores sejam imediatamente informados sobre novos *leads*, permitindo acompanhamento ágil em relação às interações com os clientes. Além disso, a notificação contribui para a centralização das informações, evitando que dados importantes fiquem dispersos ou que a gestão dependa exclusivamente de consultas ao painel de controle. A Figura 35 ilustra o modelo do e-mail enviado aos administradores, evidenciando os campos preenchidos pelo usuário

durante o cadastro e destacando como o sistema mantém a equipe de gestão informada em tempo real:

Figura 35 - E-mail novo cliente



Fonte: Autoria própria

A figura ilustra o modelo de e-mail gerado automaticamente pelo sistema e direcionado aos administradores, apresentando os principais dados do lead recém-cadastrado, como nome, e-mail e endereço.

Durante o processo de montagem do pedido e pagamento, o status do cliente no painel administrativo permanece como “Aguardando pagamento”, permitindo que os administradores identifiquem quais transações ainda estão em andamento. Esse acompanhamento em tempo real garante que a equipe de gestão tenha visibilidade imediata sobre o progresso das compras, podendo intervir ou acompanhar cada etapa conforme necessário. Após a confirmação do pagamento realizado pelo usuário, o sistema aciona o envio de um e-mail para todos os administradores, contendo as

informações detalhadas do pedido, como produtos selecionados, valores e dados do cliente. Esse mecanismo assegura que os administradores estejam sempre atualizados sobre as transações concluídas, reforçando o controle operacional e a eficiência no gerenciamento das vendas. A Figura 36 ilustra o modelo do e-mail enviado aos administradores, evidenciando os dados da compra e a confirmação de pagamento:

Figura 36 - E-mail Pagamento

O modelo do e-mail apresenta o logo da EcoStore no topo, seguido pelo título 'Compra Finalizada' e uma mensagem de confirmação. Abaixo, há seções para 'Informações do Cliente' e 'Detalhes do Pedido'. Algumas informações de contato do cliente estão mascaradas com retângulos cinza. O rodapé contém o texto 'CRM EcoStore © 2025'.

EcoStore

Compra Finalizada

Um novo pagamento foi confirmado. Seguem os detalhes do pedido:

Informações do Cliente

Nome: Lucas Foltran
Email: lucasfoltran@hotmail.com
Telefone: [Mascarado]
Endereço: [Mascarado]
Complemento: [Mascarado]

Detalhes do Pedido

Data do Pedido: 04/10/2025 19:07
Forma de Pagamento: Crédito
Total: R\$ 135,00

Itens:

Kit copos
Quantidade: 1
Preço unitário: R\$ 135,00

CRM EcoStore © 2025

Fonte: Autoria própria

Este e-mail é enviado automaticamente a todos os administradores do sistema e contém as informações completas do lead, incluindo nome, e-mail e dados de contato, além dos detalhes referentes ao pagamento efetuado. Também apresenta a lista dos itens adquiridos, com suas respectivas quantidades e valores individuais, bem como o valor total da compra.

5 CONSIDERAÇÕES FINAIS

Os resultados obtidos com o desenvolvimento do sistema CRM Operacional EcoStore demonstram a efetiva aplicação dos conceitos e metodologias propostas ao longo deste trabalho. A implementação dos fluxos de navegação, tanto do cliente desde o cadastro até a finalização da compra, quanto do administrador incluindo o acompanhamento do status dos usuários e o recebimento de notificações automáticas evidencia que o sistema atende plenamente aos objetivos iniciais definidos.

Além dos aspectos técnicos e funcionais, os resultados também demonstram contribuições relevantes no contexto da sustentabilidade. A aplicação favorece a comercialização de produtos ecologicamente responsáveis, ampliando sua visibilidade e facilitando o acesso do consumidor a alternativas sustentáveis. Ao estruturar processos comerciais de empresas desse setor, o sistema fortalece modelos de negócio comprometidos com práticas ambientais responsáveis e incentiva a consolidação do consumo consciente.

5.1 Avaliação dos Resultados e Contribuições

A plataforma possibilita o gerenciamento eficiente de clientes e pedidos, centraliza informações relevantes, garante segurança no acesso e permite o monitoramento em tempo real das transações. Dessa forma, confirma-se a viabilidade e a relevância da aplicação desenvolvida como uma solução moderna e funcional voltada à gestão de relacionamentos comerciais.

O sistema atendeu de forma satisfatória ao objetivo geral de construir uma aplicação web funcional, segura e escalável, direcionada à administração de clientes e pedidos de produtos sustentáveis. Além de demonstrar a aplicabilidade prática dos conceitos do modelo de CRM Operacional, o projeto promoveu a automação de processos internos e o acompanhamento detalhado das interações entre clientes e administradores.

A integração entre as camadas de *front-end* e *back-end* mostrou-se eficaz, assegurando desempenho adequado, consistência dos dados e uma experiência de uso intuitiva para diferentes perfis de usuário. Ademais, a adoção de tecnologias modernas, como *Angular* no desenvolvimento da interface e *ASP.NET Core* na

estrutura de serviços, contribuiu significativamente para a obtenção de uma aplicação estável, de fácil manutenção e preparada para futuras expansões.

5.2 Lições Aprendidas e Desenvolvimento de Competências

Durante o desenvolvimento deste projeto, foi possível consolidar uma série de conhecimentos técnicos e práticos relacionados ao ciclo completo de desenvolvimento de aplicações web. A utilização do *Angular* permitiu compreender de forma aprofundada conceitos de componentização, manipulação de dados via APIs e criação de interfaces dinâmicas e responsivas. No *back-end*, a aplicação de *ASP.NET Core* proporcionou o aprimoramento de habilidades na construção de serviços *RESTful*, autenticação, controle de rotas e integração com banco de dados.

Além dos aspectos técnicos, o projeto também proporcionou importantes aprendizados em relação ao planejamento de tarefas, gestão de tempo, resolução de problemas e documentação de código. Cada etapa exigiu decisões de arquitetura, testes e ajustes, contribuindo para o desenvolvimento de competências analíticas e de raciocínio lógico.

Do ponto de vista acadêmico e profissional, o trabalho reforçou a importância da interdisciplinaridade, integrando conceitos de engenharia de *software*, banco de dados, usabilidade e segurança da informação. Esse processo de construção colaborativa permitiu compreender a complexidade envolvida no desenvolvimento de sistemas reais e a relevância da aplicação dos conceitos teóricos em cenários práticos.

5.3 Recomendações para Trabalhos Futuros

Durante o processo de validação e uso do sistema, foram identificadas oportunidades de aprimoramento que podem orientar futuras versões da aplicação. Entre elas, destaca-se a possibilidade de integração de novos métodos de pagamento, como Pix e boleto bancário, ampliando as opções de transação e atendendo às preferências dos consumidores.

Outra melhoria relevante refere-se à otimização da interface para dispositivos móveis, visando garantir maior responsividade, acessibilidade e usabilidade em diferentes tamanhos de tela. Além disso, recomenda-se a ampliação das informações apresentadas no painel administrativo, permitindo uma visualização mais detalhada de

indicadores de desempenho, histórico de vendas e comportamento dos clientes, fortalecendo o caráter analítico do sistema.

Essas propostas de evolução representam não apenas ajustes técnicos, mas também avanços estratégicos, capazes de tornar o sistema ainda mais completo, eficiente e aderente às demandas do mercado contemporâneo. A continuidade desse desenvolvimento poderá fortalecer o vínculo entre inovação tecnológica e responsabilidade ambiental, consolidando o CRM Operacional EcoStore como uma ferramenta de gestão moderna e sustentável, alinhada às tendências de transformação digital e consumo consciente que caracterizam a sociedade atual.

REFERÊNCIAS

BECKER, L. G. Como a tecnologia impulsiona o sucesso comercial? **A importância do CRM na gestão de vendas**. Saber Humano, [Rio Grande do Sul], Edição Especial: Cadernos de Iniciação Científica – “Eu vejo, eu faço”, p. 131-149, fev. 2024.

Disponível em: <https://saberhumano.emnuvens.com.br/sh/article/view/670/617>. Acesso em: 20 mar. 2025.

CONCEIÇÃO, W. S.; RIPPEL, E. V. **O impacto do CRM na gestão de relacionamento com o cliente**. Revista Novos Desafios, Araguaína, v. 2, n. 2, p. 19- 33, 2022. Disponível em:

<http://novosdesafios.inf.br/index.php/revista/article/view/52>. Acesso em: 16 mar. 2025.

DAS, S.; HASSAN, H. M. K. Impact of sustainable supply chain management and customer relationship management on organizational performance. **International Journal of Productivity and Performance Management**, [S.l], v. 70, n. 8, p. 2059-2080, ago. 2021. DOI: 10.1108/IJPPM-08-2020-0441. Disponível em:

<https://www.researchgate.net/publication/354171052>. Acesso em: 11 out. 2025.

HOFFMANN, S.; PINTO, L. A.; URIARTE, L. R. **Análise comparativa entre as tecnologias de front-end React, Angular e Vue**. Saber Humano, [Rio Grande do Sul], Edição Especial: Cadernos de Iniciação Científica – “Eu vejo, eu faço”, p. 131–149, nov. 2022.

Disponível em: <https://dalfovo.com/ojs/index.php/reis/article/view/398/379>. Acesso em: 15 out. 2025.

LESSA, C. I. B. S. **A importância da sustentabilidade ambiental e da reciclagem na sociedade contemporânea: uma revisão bibliográfica**. 2020. Trabalho de Conclusão de Curso (Graduação em Ciências Biológicas) – Instituto de Ciências Biológicas e da Saúde, Universidade Federal de Alagoas, Maceió, 2020. Disponível em: <https://www.repositorio.ufal.br/handle/123456789/14943>. Acesso em: 11 out. 2025.

LIMA, G. T. **Desenvolvimento da aplicação web EleTrip para locação de carros elétricos utilizando Angular**. 2023. Relatório Técnico (Bacharelado em Ciência da Computação) – Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, São João da Boa Vista, 2023. Disponível em:

<https://repositorio.ifsp.edu.br/items/442c5089-e08a-401a-adfe-c71e1ed520c2>. Acesso em: 17 out. 2025.

LUCIANO, J.; ALVES, W. J. B. **Padrão de Arquitetura MVC: Model-View-Controller**. Revista EPeQ Fafibe, Bebedouro, SP, v. 1, n. 3, 2011. Disponível em: <https://unifafibe.com.br/revistasonline/arquivos/revistaepeqfafibe/sumario/20/16112011142249.pdf>. Acesso em: 31 ago. 2025.

MARIANO, A.; CALVINO, A.; MELLO, A. N. **Experiências que deixam marcas - Volume 3: Estratégias e ferramentas do CRM para uma melhor experiência do cliente.**

MATIELLO, R. **Estudo de caso da aplicação do CRM em empresas.** 2020. 53 f. Trabalho de Conclusão de Curso (Graduação em Engenharia de Produção) – Universidade Federal de Ouro Preto, Ouro Preto, 2020. Disponível em: https://monografias.ufop.br/bitstream/35400000/4442/1/MONOGRAFIA_EstudoCasoAplica%C3%A7%C3%A3o.pdf. Acesso em: 17 mar. 2025.

OLIVEIRA, A. V. M.; OLIVEIRA, M. R. B.; PAIVA, L. F. R. **Plataforma web para contratação de serviços.** Uberaba: Universidade de Uberaba, 2023. Disponível em: <https://dspace.uniube.br/bitstream/123456789/2676/1/Plataforma%20web%20para%20contratação%20de%20serviços.pdf>. Acesso em: 31 ago. 2025.

OLIVEIRA, G. A. B. **CRM como ferramenta estratégica comercial: Engenharia Organizacional.** 2024. 39 f. Trabalho de Conclusão de Curso (Graduação em Engenharia de Produção) – Universidade Federal de Mato Grosso do Sul, Campo Grande, MS, 2024. Disponível em: <https://repositorio.ufms.br/jspui/retrieve/4fbf7390-7876-416d-9e4c-c2b582398531/14528.pdf>. Acesso em: 20 mar. 2025.

ORSATTO, G.; MACHADO, L. **Uma análise dos TCCs sobre CRM na Graduação UFSC.** 2022. 36 f. Trabalho de Conclusão de Curso (Graduação em Administração) – Universidade Federal de Santa Catarina, Florianópolis, 2022. Disponível em: <https://repositorio.ufsc.br/bitstream/handle/123456789/255667/TCC%20-%20Uma%20an%C3%A1lise%20dos%20TCCs%20sobre%20CRM%20na%20Gradua%C3%A7%C3%A3o%20UFSC.pdf?sequence=3&isAllowed=y>. Acesso em: 17 mar. 2025.

PAVANELI, T. H. **Desenvolvimento de uma API e uma aplicação web para auxiliar na análise de dados providos por aplicação móvel.** 2023. Trabalho de Conclusão de Curso (Bacharelado em Sistemas de Informação) – Faculdade de Computação, Universidade Federal de Uberlândia, Uberlândia, 2023. Disponível em: <https://repositorio.ufu.br/handle/123456789/37886>. Acesso em: 17 out. 2025.

RENATA, C. P. **Os desafios e vantagens da sustentabilidade empresarial.** Anápolis: Instituto Metropolitano de Educação e Cultura LTDA F.A.M.A., 2021. Disponível em: <https://repositorio.faculdefama.edu.br/xmlui/handle/123456789/98>. Acesso em: 11 out. 2025.

SILVEIRA, J. S. **Gestão de relacionamento com o cliente (CRM): ferramenta para promover a satisfação e fidelização de seus clientes no mercado atual.** 2022. 48 f. Trabalho de Conclusão de Curso (Graduação em Administração) – Unidade de Ensino Superior Dom Bosco, São Luís, 2022. Disponível em: <http://repositorio.undb.edu.br/bitstream/areas/754/1/JOELSON%20SANTOS%20SILVEIRA.pdf>. Acesso em: 21 mar. 2025.

SIMÕES, J. C. R. **A importância do CRM e o seu impacto no relacionamento com os clientes.** 2021. 131 f. Trabalho de Conclusão de Curso (Graduação em Contabilidade e Administração) – Instituto Politécnico de Coimbra, Instituto Superior de Contabilidade e Administração de Coimbra, Coimbra, 2021. Disponível em: <https://comum.rcaap.pt/entities/publication/c8be4fa0-c4a0-44af-b4c3-a5699027e6f5>. Acesso em: 22 mar. 2025.

SOUZA, A. et al. **A Ciência do CRM: A Bíblia do Marketing de Relacionamento e da Experiência do Cliente: do Tradicional ao Digital.** São Paulo: Novatec, 2009.

SOUZA, L. V.; BOER, M. T. **Estudo comparativo entre os modelos de banco de dados NoSQL e SQL.** 2023. Trabalho de Graduação (Tecnólogo em Sistemas para Internet) – Faculdade de Tecnologia Professor José Camargo, Fatec Jales, 2023. Disponível em: https://ric.cps.sp.gov.br/bitstream/123456789/16325/1/sistemas_para_internet_2023_1_larissa_vital_de_souza_estudo_comparativo_entre_os_modelos_de_banco_de_dados.pdf. Acesso em: 31 ago. 2025.

ZANATTA, L. H. **A implementação do CRM: tecnologia para potencializar a gestão de vendas, relacionamentos e resultados.** 2024. 64 f. Trabalho de Conclusão de Curso (Graduação em Administração) – FAT – Faculdade e Escola, Tapejara, RS, 2024. Disponível em: <https://fatrs.com.br/arq/biblioteca/356a192b7913b04c54574d18c28d46e6395428ab/tcc/b3f0c7f6bb763af1be91d9e74eabfeb199dc1f1f/1.7387673395e+12.pdf>. Acesso em: 19 mar. 2025.

APÊNDICE A – REPOSITÓRIO DO PROJETO

Este apêndice apresenta o endereço do repositório *Git* que contém todo o código-fonte desenvolvido para o sistema CRM Operacional EcoStore, incluindo os módulos de *front-end* e *back-end*, arquivos de configuração, documentação e instruções de execução do projeto.

O repositório foi hospedado em plataforma pública para fins acadêmicos, possibilitando o acesso ao conteúdo completo do sistema e às versões implementadas ao longo do desenvolvimento.

Link do repositório:

<https://github.com/Lucas-Foltran/CRM-EcoStore>

